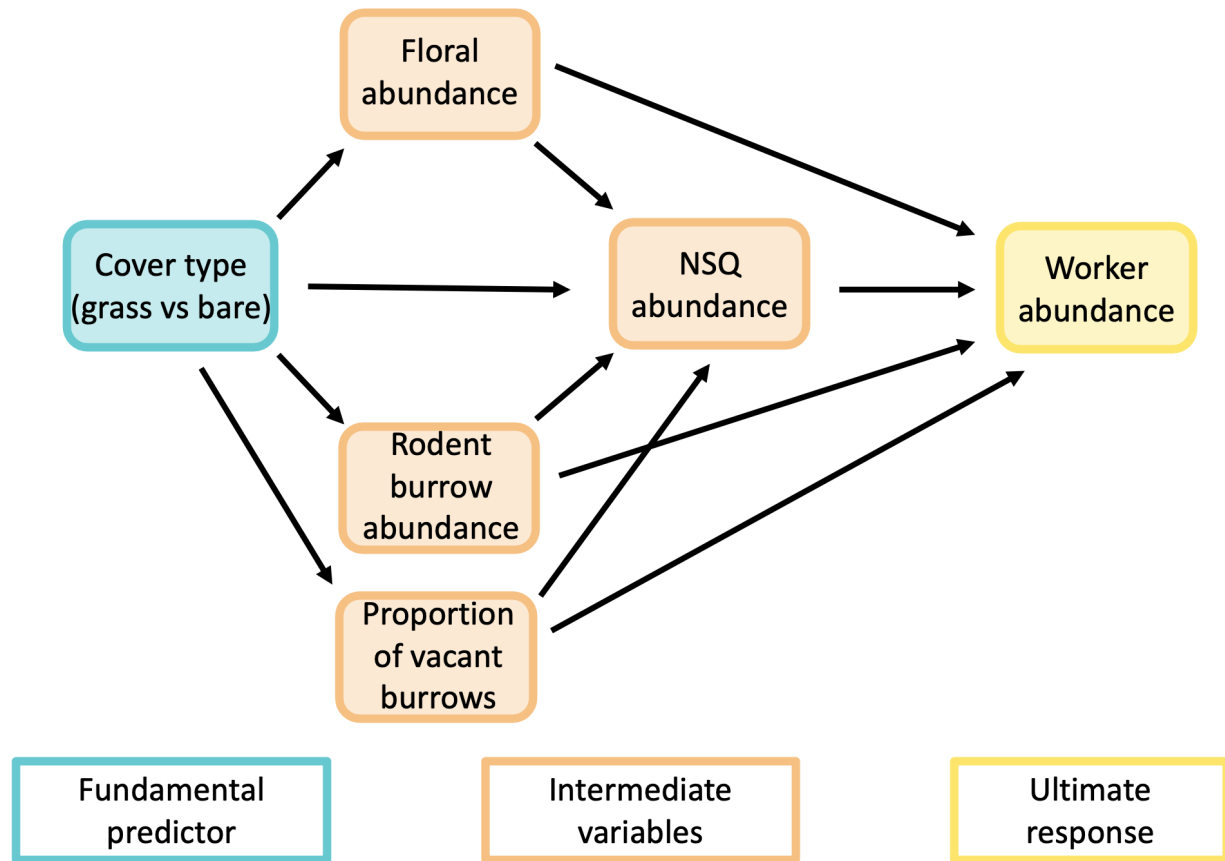


ch1_glm_fullRE

Predicted path diagram



Load and clean data

```
#load packages
library(tidyverse) #data manipulation, visualization etc. (dplyr, ggplot2, tidyr)

## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr    1.5.1
## v ggplot2    3.5.2      v tibble     3.3.0
## v lubridate  1.9.4      v tidyr      1.3.1
## v purrr      1.0.4
```

```
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag() masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(DHARMA)      #diagnostic tools for mixed regression models
```

```
## This is DHARMA 0.4.7. For overview type '?DHARMA'. For recent changes, type news(package = 'DHARMA')
```

```
library(ordinal)     #models for ordinal outcomes (e.g., cumulative link models)
```

```
##
## Attaching package: 'ordinal'
##
## The following object is masked from 'package:dplyr':
##
## slice
```

```
library(MASS)        #polr
```

```
##
## Attaching package: 'MASS'
##
## The following object is masked from 'package:dplyr':
##
## select
```

```
library(glmmTMB)     #fits generalized linear mixed models, including zero-inflated and overdispersed models
library(car)         #tools for regression diagnostics and hypothesis tests
```

```
## Loading required package: carData
##
## Attaching package: 'car'
##
## The following object is masked from 'package:dplyr':
##
## recode
##
## The following object is masked from 'package:purrr':
##
## some
```

```
library(emmeans)     #calculates predicted results from models
```

```
## Welcome to emmeans.
## Caution: You lose important information if you filter this package's results.
## See '? untidy'
```

```

#read in data
data <- read.csv("06_cleandata/combined_plot_data_ch1.csv")

#change character columns to factors
surveys <- data %>%
  mutate(
    site_id = as.factor(site_id),
    round = as.factor(round),
    plot_id = as.factor(plot_id),
    bee_date = as.Date(bee_date),
    bee_season_type = as.factor(bee_season_type),
    area = as.factor(area),
    cover_type = as.factor(cover_type))

#update cover type names
levels(surveys$cover_type) <- c("bare", "herbaceous")

#scale continuous predictors
surveys$julian.date_scaled <- as.numeric(scale(surveys$julian.date))
surveys$bare_perc_scaled <- as.numeric(scale(surveys$bare_perc))
surveys$grass_perc_scaled <- as.numeric(scale(surveys$grass_perc))
surveys$forb_perc_scaled <- as.numeric(scale(surveys$forb_perc))
surveys$grass_forb_perc_scaled <- as.numeric(scale(surveys$grass_forb_perc))
surveys$grass_h_scaled <- as.numeric(scale(surveys$grass_h))
surveys$forb_h_scaled <- as.numeric(scale(surveys$forb_h))
surveys$floral_perc_scaled <- as.numeric(scale(surveys$floral_perc))
surveys$grass_forb_floral_perc_scaled <- as.numeric(scale(surveys$grass_forb_floral_perc))
surveys$bombus_floral_perc_scaled <- as.numeric(scale(surveys$bombus_floral_perc))
surveys$flood_perc_scaled <- as.numeric(scale(surveys$flood_perc))
surveys$dry_root_perc_scaled <- as.numeric(scale(surveys$dry_root_perc))
surveys$bryo_perc_scaled <- as.numeric(scale(surveys$bryo_perc))
surveys$bare_flood_dry_bryo_perc_scaled <- as.numeric(scale(surveys$bare_flood_dry_bryo_perc))
surveys$floral_richness_scaled <- as.numeric(scale(surveys$floral_richness))
surveys$bombus_floral_richness_scaled <- as.numeric(scale(surveys$bombus_floral_richness))
surveys$bombus_floral_abun_scaled <- as.numeric(scale(surveys$bombus_floral_abun))
surveys$bombus_floral_abun_nv_scaled <- as.numeric(scale(surveys$bombus_floral_abun_nv))
surveys$nest_searching_queens_scaled <- as.numeric(scale(surveys$nest_searching_queens))
surveys$workers_scaled <- as.numeric(scale(surveys$workers))
surveys$forage_vaco_scaled <- as.numeric(scale(surveys$forage_vaco))
surveys$num_burrows_scaled <- as.numeric(scale(surveys$num_burrows))
surveys$num_active_burrows_scaled <- as.numeric(scale(surveys$num_active_burrows))
surveys$num_inactive_burrows_scaled <- as.numeric(scale(surveys$num_inactive_burrows))
surveys$num_active_dm_scaled <- as.numeric(scale(surveys$num_active_dm))
surveys$num_dm_size_scaled <- as.numeric(scale(surveys$num_dm_size))
surveys$num_inactive_dm_scaled <- as.numeric(scale(surveys$num_inactive_dm))
surveys$prop_inactive_burrows_scaled <- as.numeric(scale(surveys$prop_inactive_burrows))
surveys$num_runway_burrows_scaled <- as.numeric(scale(surveys$num_runway_burrows))

#create variable for proportion of vacant burrows (# inactive / total)
surveys$prop_inactive <- surveys$num_inactive_burrows / (surveys$num_inactive_burrows + surveys$num_act
#scale
surveys$prop_inactive_scaled <- as.numeric(scale(surveys$prop_inactive))
#check correlation

```

```
cor(
  surveys$prop_inactive,
  surveys$num_burrows,
  use = "complete.obs"
)
```

```
## [1] 0.06838928
```

##r = 0.06. This means that the vacancy probability and rodent burrow abundance are largely independent

```
#filter to only include field data
field_data <- surveys %>%
  filter(plot_id != "d")
```

Rodent burrows vs field management (%grass+forb)

Since the number of rodent burrows was count data, I knew I had to use either a poisson or negative binomial GLM family. I started with the simplest family (poisson) first. I didn't include julian.date as a covariate in this model since I assumed that the number of rodent burrows remains relatively consistent over the course of a season (hence why I only sampled each farm once).

Steps

- 1) Run model with site nested within area as random effect. I did not include plot as a random effect structure since there was only one observation per plot for rodent burrow data.

- best fit = 1|site_id -> **area not included**

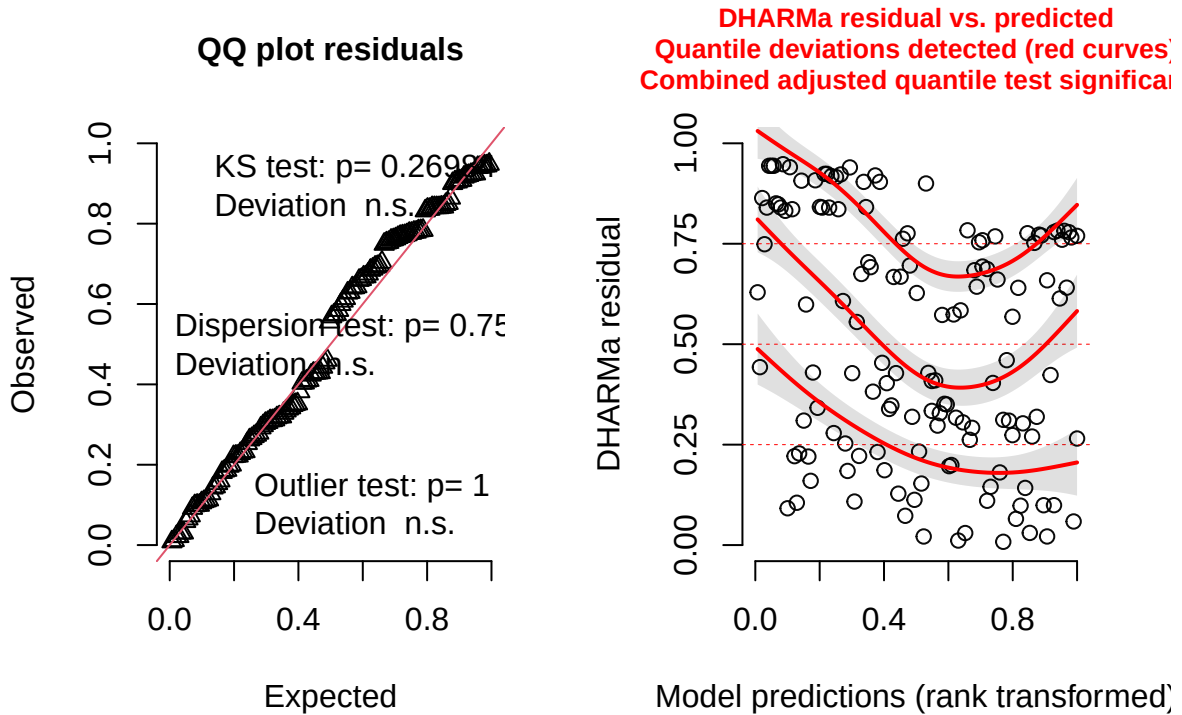
```
burrows_full <- glmmTMB(num_burrows ~ grass_forb_perc_scaled +
  (1|area/site_id),
  family = poisson,
  data = field_data)
```

- 2) Check diagnostics of the model using DHARMA package.

- Combined adjusted quantile test significant

```
#simulate residuals
residuals_burrows <- simulateResiduals(fittedModel = burrows_full, plot = TRUE)
```

DHARMa residual



3) Try negative binomial instead and check diagnostics.

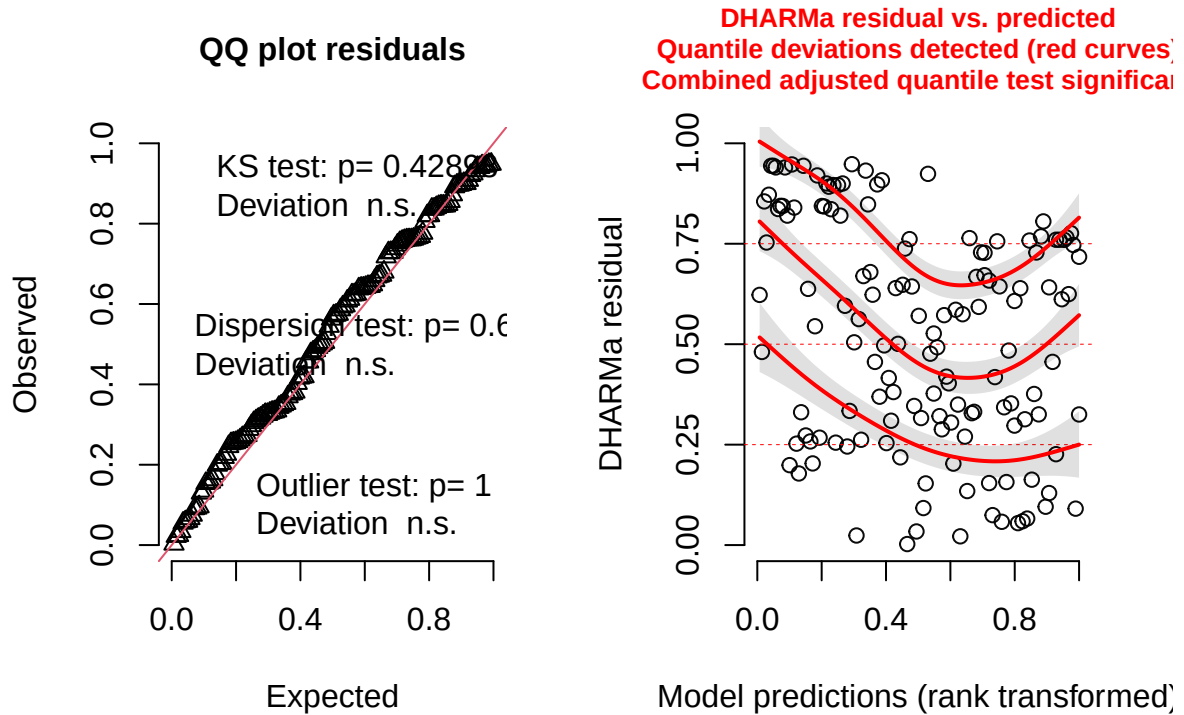
- Combined adjusted quantile test still significant.

```
burrows_negBin <- glmmTMB(num_burrows ~ grass_forb_perc_scaled +
  (1|area/site_id),
  family = nbinom2,
  data = field_data)

#simulate residuals
residuals_burrowsy_negBin <- simulateResiduals(fittedModel = burrows_negBin, plot = TRUE)

## Warning in newton(lsp = lsp, X = G$X, y = G$y, Eb = G$Eb, UrS = G$UrS, L = G$L,
## : Fitting terminated with step failure - check results carefully
```

DHARMa residual



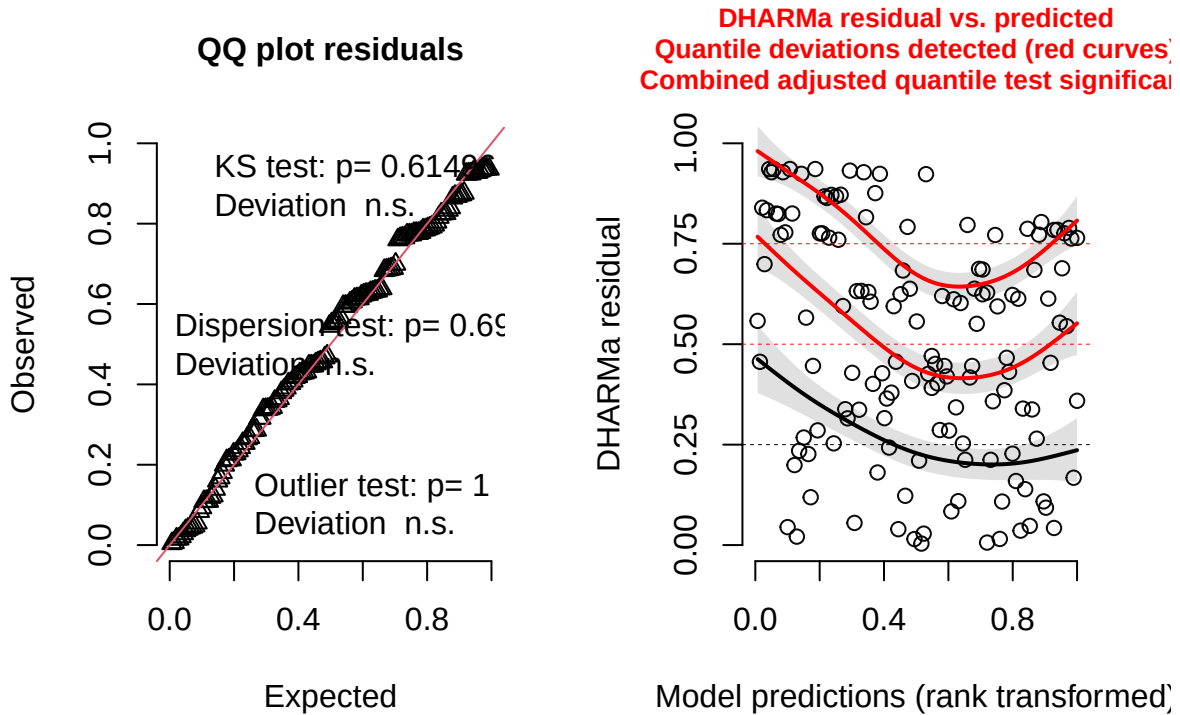
4) Try zero-inflated poisson and check diagnostics.

- combined adjusted quantile test still significant.

```
burrows_zi_pois <- glmmTMB(num_burrows ~ grass_forb_perc_scaled +
  (1|area/site_id),
  ziformula = ~ grass_forb_perc_scaled +
  (1|area/site_id),
  family = poisson,
  data = field_data)

#simulate residuals
residuals_burrows_zi <- simulateResiduals(fittedModel = burrows_zi_pois, plot = TRUE)
```

DHARMa residual



5) Try zero-inflated negative binomial and check diagnostics.

- combined adjusted quantile test still significant.

```
burrows_zi_nbin <- glmmTMB(num_burrows ~ grass_forb_perc_scaled +
  (1|area/site_id),
  ziformula = ~ grass_forb_perc_scaled +
  (1|area/site_id),
  family = nbinom2,
  data = field_data)
```

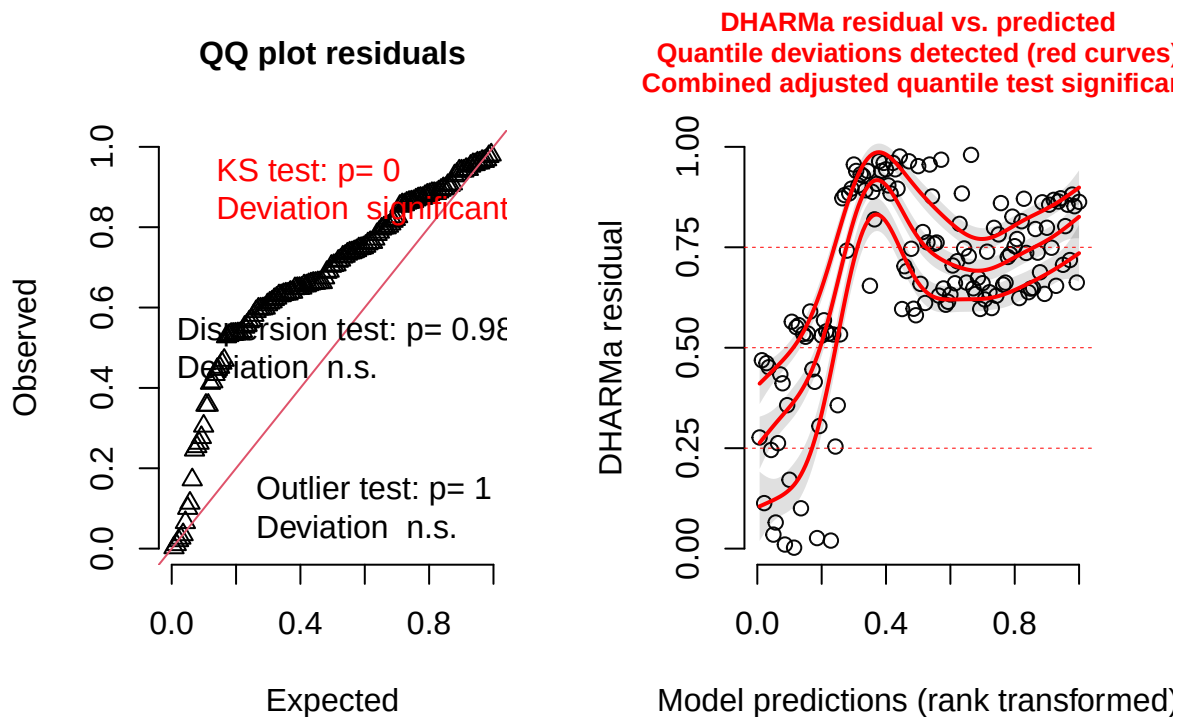
```
## Warning in (function (start, objective, gradient = NULL, hessian = NULL, :
## NA/NaN function evaluation
## Warning in (function (start, objective, gradient = NULL, hessian = NULL, :
## NA/NaN function evaluation
## Warning in (function (start, objective, gradient = NULL, hessian = NULL, :
## NA/NaN function evaluation
## Warning in (function (start, objective, gradient = NULL, hessian = NULL, :
## NA/NaN function evaluation
```

```
## Warning in finalizeTMB(TMBStruc, obj, fit, h, data.tmb.old): Model convergence
## problem; non-positive-definite Hessian matrix. See vignette('troubleshooting')
```

```
## Warning in finalizeTMB(TMBStruc, obj, fit, h, data.tmb.old): Model convergence
## problem; false convergence (8). See vignette('troubleshooting'),
## help('diagnose')
```

```
#simulate residuals
residuals_burrows_zi_nbin <- simulateResiduals(fittedModel = burrows_zi_nbin, plot = TRUE)
```

DHARMA residual



#significant KS test and combined adjusted quantile test

- 6) Stick with poisson but try using categorical cover_type as predictor instead of % grass/forb cover and try different random effect structures.
 - models using continuous percent grass/forb cover as a predictor showed poor fit and significant residual patterns, likely due to the small sample size ($n = 24$). Because sites were grouped into two distinct habitat types—grassy plots with high vegetation cover and bare plots with little or no cover—the variable was reclassified as a categorical factor (“grass” vs. “bare”). This simplified model structure improved residual diagnostics and provided a clearer ecological interpretation of differences in burrow abundance between habitat types.

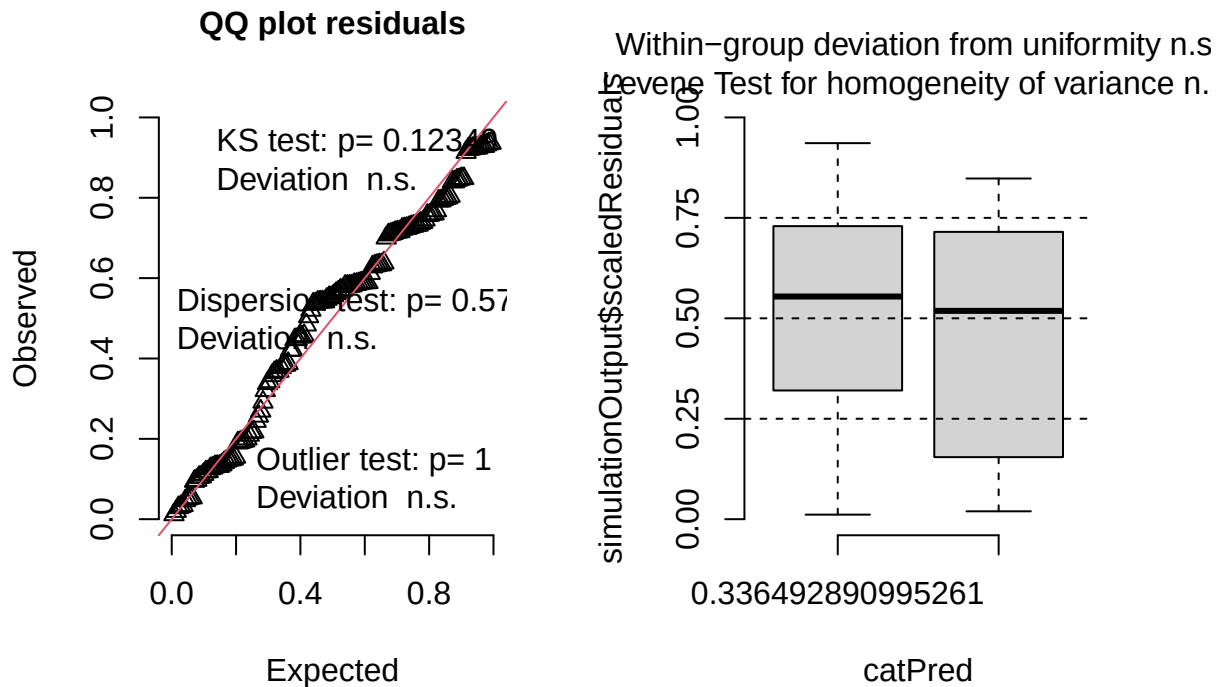
```
burrows_cover_full <- glmmTMB(num_burrows ~ cover_type +
  (1|area/site_id),
  family = poisson,
  data = field_data)
```

7. Check diagnostics

- significant within-group deviations from uniformity


```
#simulate residuals
residuals_burrows_cover_full <- simulateResiduals(fittedModel = burrows_cover_full, plot = TRUE)
```

DHARMA residual



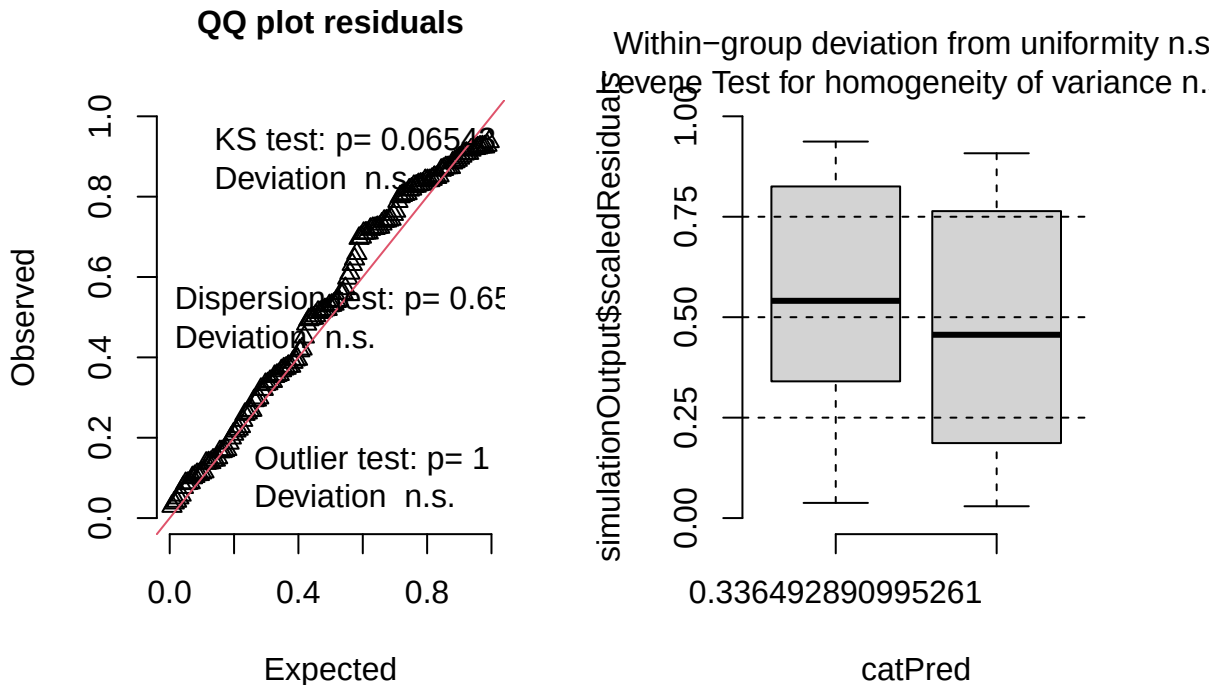
8. Try simpler random effect structure (site-only) and re-check diagnostics

- No significant problems detected.

```
burrows_cover_negBin <- glmmTMB(num_burrows ~ cover_type +
                                (1|area/site_id),
                                family = nbinom2,
                                data = field_data)

#simulate residuals
residuals_burrows_cover_negBin <- simulateResiduals(fittedModel = burrows_cover_negBin, plot = TRUE)
```

DHARMA residual



9. Plot results

```
summary(burrows_cover_negBin)
```

```
## Family: nbinom2 ( log )
## Formula: num_burrows ~ cover_type + (1 | area/site_id)
## Data: field_data
##
##      AIC      BIC    logLik -2*log(L)  df.resid
##    843.9    858.6   -416.9    833.9     135
##
## Random effects:
##
## Conditional model:
## Groups      Name      Variance Std.Dev.
## site_id:area (Intercept) 1.605    1.267
## area         (Intercept) 2.416    1.554
## Number of obs: 140, groups: site_id:area, 12; area, 6
##
## Dispersion parameter for nbinom2 family (): 2.9
##
## Conditional model:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)      0.6125   0.8501   0.721   0.4712
## cover_typeherbaceous 2.0167   0.7710   2.616   0.0089 **
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Back transform effect size to response scale

```
exp(2.0167) ##7.5
```

```
## [1] 7.513489
```

Plots

Number of rodent burrows vs cover type

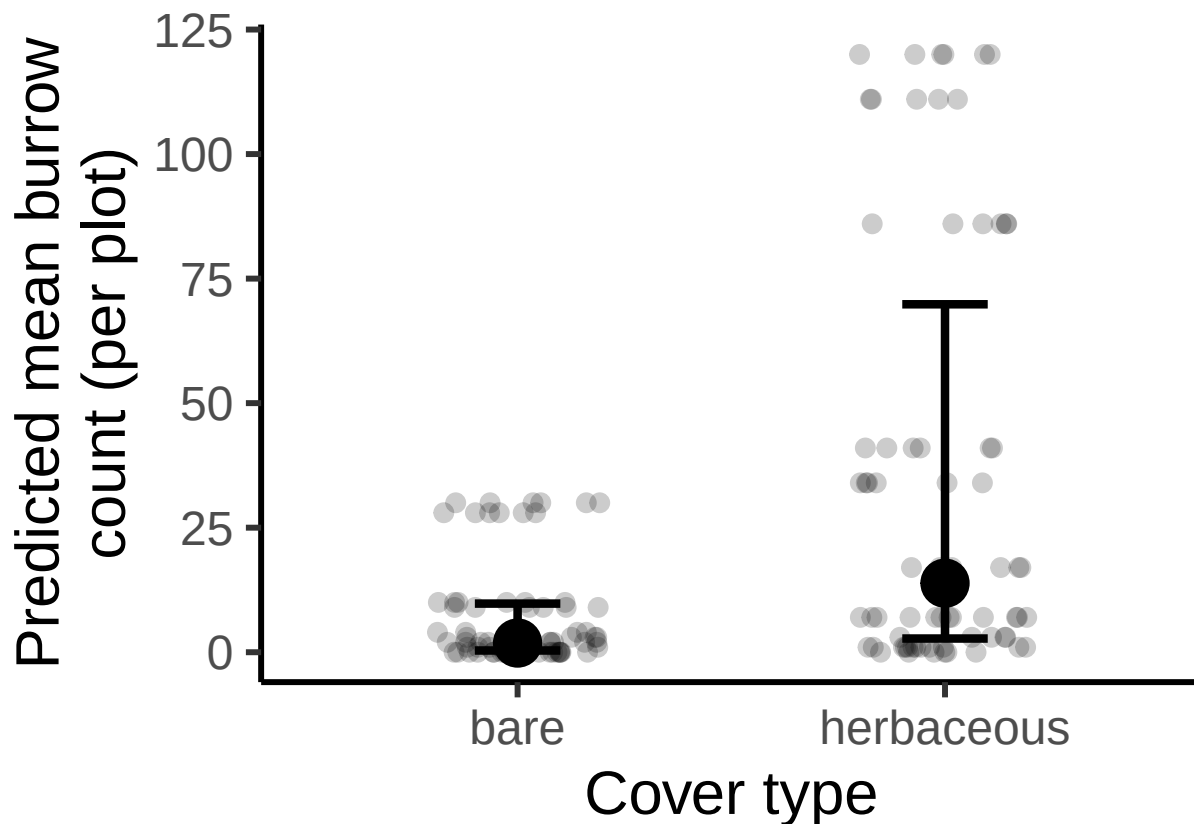
```
#create new data grid
rodent_data_cover<- expand.grid(
  cover_type = unique(field_data$cover_type)
)

# Get predicted probabilities and standard errors
pred_rodent_cover <- predict(burrows_cover_negBin,
                             newdata = rodent_data_cover,
                             type = "link",
                             se.fit = TRUE,
                             re.form = NA)

# Combine predictions with data frame
pred_rodent_cover_df <- cbind(rodent_data_cover,
                              fit_log = pred_rodent_cover$fit,
                              se_log = pred_rodent_cover$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_rodent_cover_df <- pred_rodent_cover_df %>%
  mutate(
    fit = exp(fit_log),
    lower = exp(fit_log - 1.96 * se_log),
    upper = exp(fit_log + 1.96 * se_log)
  )

ggplot(pred_rodent_cover_df, aes(x = cover_type, y = fit)) +
  geom_jitter(
    data = field_data,
    aes(x = cover_type, y = num_burrows),
    width = 0.2,
    height = 0,
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_point(size = 8) +
  geom_errorbar(aes(ymin = lower, ymax = upper), width = 0.2, position = position_dodge(0.6), linewidthh
  labs(x = "Cover type",
       y = "Predicted mean burrow\ncount (per plot)") +
  theme_classic(base_size = 23)
```



Findings

Neither grass + forb % nor julian date had significant effects on rodent burrow abundance. However, cover type showed a marginally insignificant trend ($p=0.0839$), where grassy sites had a higher abundance of burrows.

Rodent burrow vacancy analysis

I want to determine what influences the proportion of burrows that are inactive (available as nest sites for bumble bees) across my sites.

Steps

- 1) Try different random effect structures in a binomial model. Compare alternative random effect structures using maximum likelihood (ML) via AIC. I did not include plot as a random effect structure since there was only one observation per plot for rodent burrow data.

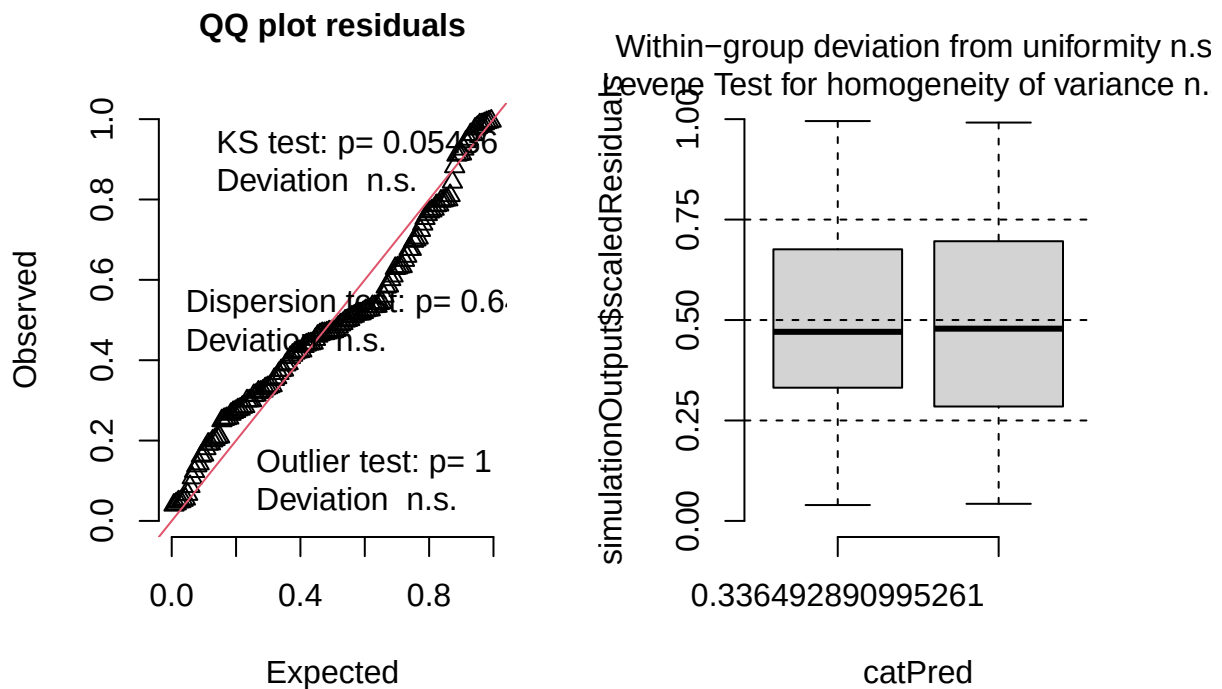
```
burrows_vacancy_full <- glmmTMB(
  cbind(num_inactive_burrows, num_active_burrows) #tells the model that successes = number of inactive burrows
  ~ cover_type + (1|area/site_id),
  family = binomial(link = "logit"),
  data = field_data)
```

2) Check diagnostics

- The diagnostic plot looks good.

```
#simulate residuals
residuals_burrows_vacancy_full <- simulateResiduals(fittedModel = burrows_vacancy_full, plot = TRUE)
```

DHARMA residual



```
##KS test significant
```

3. Print results

```
summary(burrows_vacancy_full)
```

```
## Family: binomial ( logit )
## Formula:
## cbind(num_inactive_burrows, num_active_burrows) ~ cover_type +
## (1 | area/site_id)
## Data: field_data
##
##      AIC      BIC    logLik -2*log(L)  df.resid
##    465.2    477.0   -228.6    457.2      136
##
## Random effects:
##
```

```
## Conditional model:
##   Groups      Name      Variance Std.Dev.
## site_id:area (Intercept) 1.587e+00 1.2597964
## area         (Intercept) 2.550e-08 0.0001597
## Number of obs: 140, groups: site_id:area, 12; area, 6
##
## Conditional model:
##               Estimate Std. Error z value Pr(>|z|)
## (Intercept)      0.006842   0.620446   0.011   0.991
## cover_typeherbaceous -0.358814   0.831891  -0.431   0.666
```

Plots

Proportion of vacant burrows vs cover type

```
#create new data grid
vacancy_data_cover<- expand.grid(
  cover_type = unique(field_data$cover_type)
)

# Get predicted probabilities and standard errors
pred_vacancy_cover <- predict(burrows_vacancy_full,
                              newdata = vacancy_data_cover,
                              type = "link",
                              se.fit = TRUE,
                              re.form = NA)

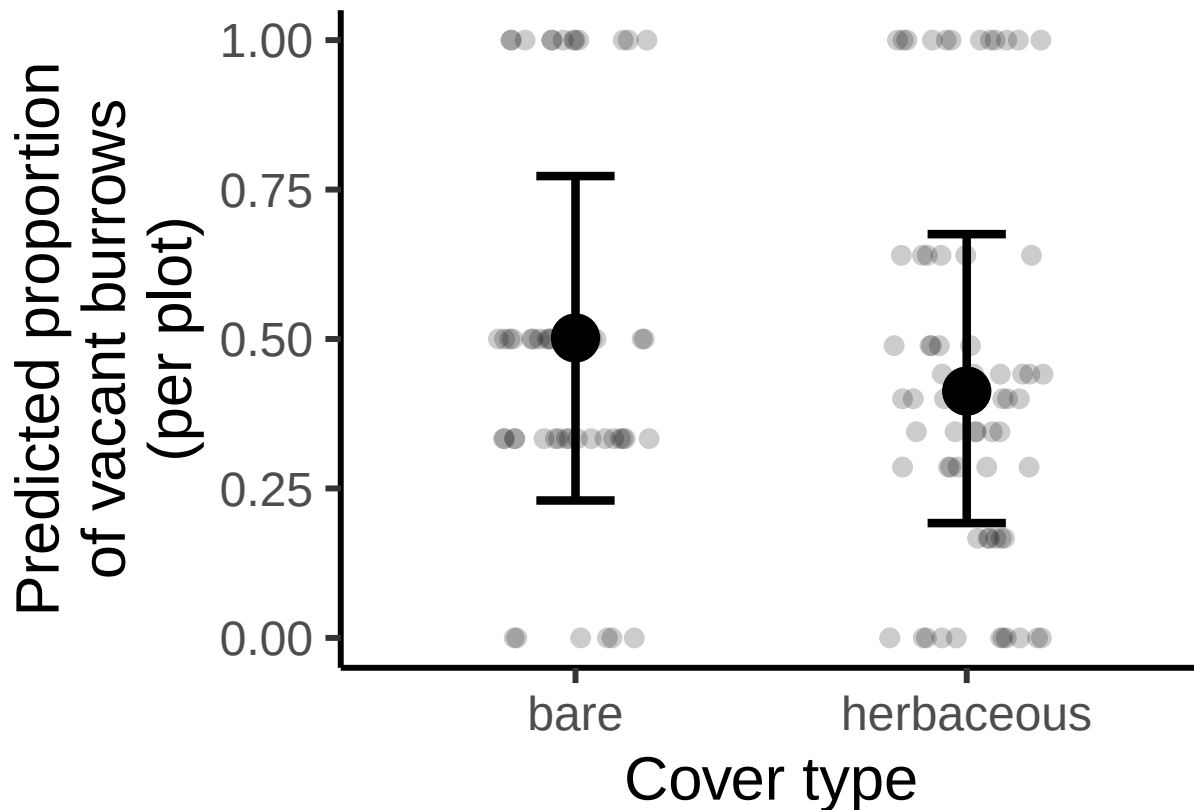
# Combine predictions with data frame
pred_vacancy_cover_df <- cbind(vacancy_data_cover,
                               fit_logit = pred_vacancy_cover$fit,
                               se_logit = pred_vacancy_cover$se.fit)

#calculate predictions and confidence intervals on response scale (not logit scale)
pred_vacancy_cover_df <- pred_vacancy_cover_df %>%
  mutate(
    fit = plogis(fit_logit),
    lower = plogis(fit_logit - 1.96 * se_logit),
    upper = plogis(fit_logit + 1.96 * se_logit)
  )

ggplot(pred_vacancy_cover_df, aes(x = cover_type, y = fit)) +
  geom_jitter(
    data = field_data,
    aes(x = cover_type, y = prop_inactive),
    width = 0.2,
    height = 0,
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_point(size = 8) +
  geom_errorbar(aes(ymin = lower, ymax = upper), width = 0.2, position = position_dodge(0.6), linewidth
  labs(x = "Cover type",
```

```
y = "Predicted proportion\nof vacant burrows\n(per plot)" +
theme_classic(base_size = 23)
```

```
## Warning: Removed 24 rows containing missing values or values outside the scale range
## ('geom_point()').
```



Findings

There was not a significant difference in burrow vacancy probability between grassy and bare sites (log-odds difference = -0.36 , $SE = 0.83$, $p = 0.67$). Burrow vacancy probability was approximately 50% across sites, meaning that roughly one in two burrows at blueberry farms in this study were available as potential nest sites for bumble bees.

Floral abundance vs field management (%grass+forb)

Use cumulative link model aka ordinal logistic regression because bombus-relevant floral abundance has:

- discrete levels (0-5 scale) representing binned counts of blooms
- ordered categories (higher numbers represent more blooms)
- unequal intervals between categories

Steps

- 1) Run CLM with random effect structure (full = 1|area/site/plot)

```
floral_full <- clmm(as.factor(bombus_floral_abun) ~ cover_type +  
                    julian.date_scaled +  
                    (1|area/site_id/plot_id),  
                    link = "logit",  
                    data = field_data)
```

2. Run final model and look at output

```
summary(floral_full)  
  
## Cumulative Link Mixed Model fitted with the Laplace approximation  
##  
## formula: as.factor(bombus_floral_abun) ~ cover_type + julian.date_scaled +  
##          (1 | area/site_id/plot_id)  
## data:    field_data  
##  
## link threshold nobs logLik AIC      niter      max.grad cond.H  
## logit flexible  140  -211.59 441.17 735(759) 8.90e-05 3.9e+01  
##  
## Random effects:  
## Groups              Name              Variance Std.Dev.  
## plot_id:(site_id:area) (Intercept) 0.000e+00 0.000e+00  
## site_id:area           (Intercept) 1.887e-11 4.344e-06  
## area                   (Intercept) 0.000e+00 0.000e+00  
## Number of groups: plot_id:(site_id:area) 24, site_id:area 12, area 6  
##  
## Coefficients:  
##              Estimate Std. Error z value Pr(>|z|)  
## cover_typeherbaceous  0.02646    0.30414   0.087   0.931  
## julian.date_scaled    -0.23118    0.15268  -1.514   0.130  
##  
## Threshold coefficients:  
##      Estimate Std. Error z value  
## 0|1  -1.3986    0.2610  -5.359  
## 1|2  -0.9291    0.2414  -3.849  
## 2|3  -0.1853    0.2272  -0.815  
## 3|4   0.6192    0.2332   2.656
```

Plots

Vegetation

```
emm_floral <- emmeans(  
  floral_full,  
  ~ cover_type,  
  at = list(julian.date_scaled = mean(field_data$julian.date_scaled, na.rm = TRUE)),  
  mode = "mean.class" ## The expected floral abundance category (mean ordinal score)
```



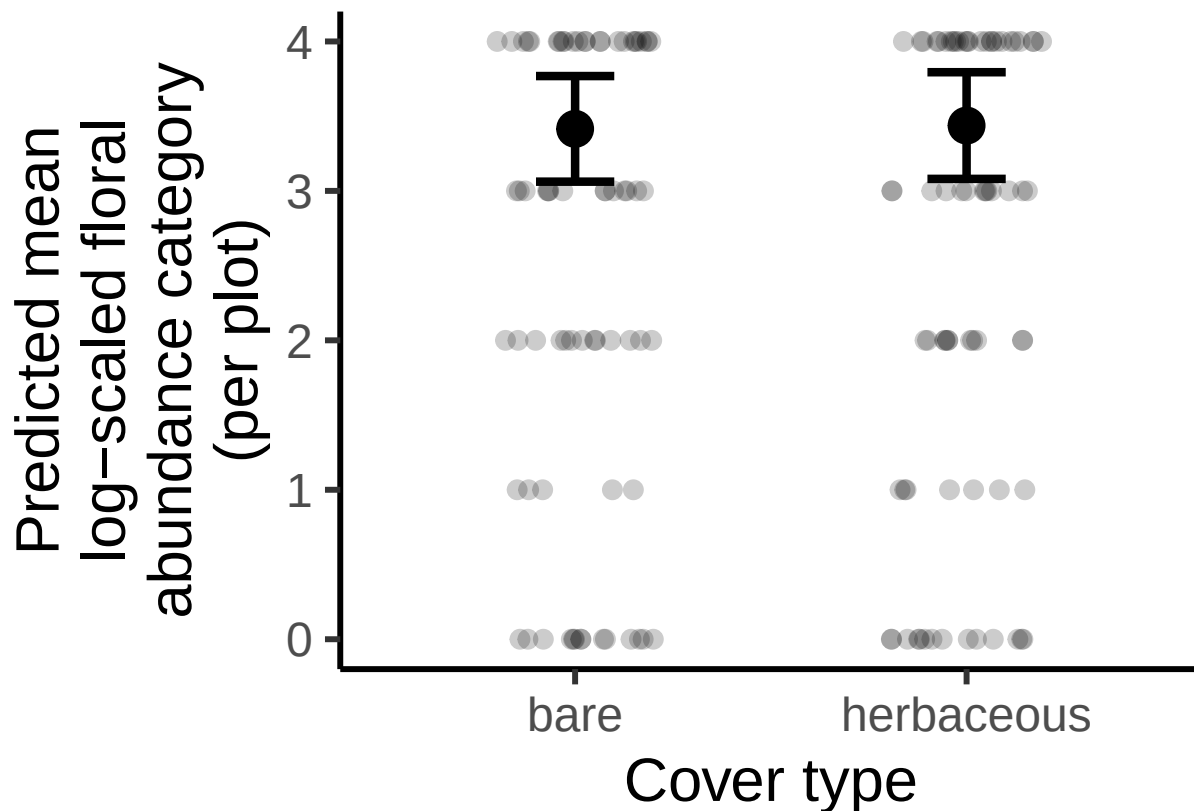
```

)

#convert to dataframe
emm_floral_df <- as.data.frame(emm_floral)

#plot
ggplot(emm_floral_df, aes(x = cover_type, y = mean.class)) +
  geom_jitter(
    data = field_data,
    aes(x = cover_type, y = bombus_floral_abun),
    width = 0.2,
    height = 0,
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_point(size = 6) +
  geom_errorbar(
    aes(ymin = asymp.LCL, ymax = asymp.UCL),
    width = 0.2,
    linewidth = 1.5
  ) +
  labs(
    x = "Cover type",
    y = "Predicted mean\nlog-scaled floral\nabundance category\nn(per plot)"
  ) +
  theme_classic(base_size = 23)

```



Julian date

```
emm_floral_date <- emmeans(
  floral_full,
  ~ julian.date_scaled,
  at = list(cover_type = "herbaceous",
            julian.date_scaled = seq(
              min(field_data$julian.date_scaled, na.rm = TRUE),
              max(field_data$julian.date_scaled, na.rm = TRUE),
              length.out = 50)),
  mode = "mean.class" ## The expected floral abundance category (mean ordinal score)
)

#convert to dataframe
emm_floral_date_df <- as.data.frame(emm_floral_date)

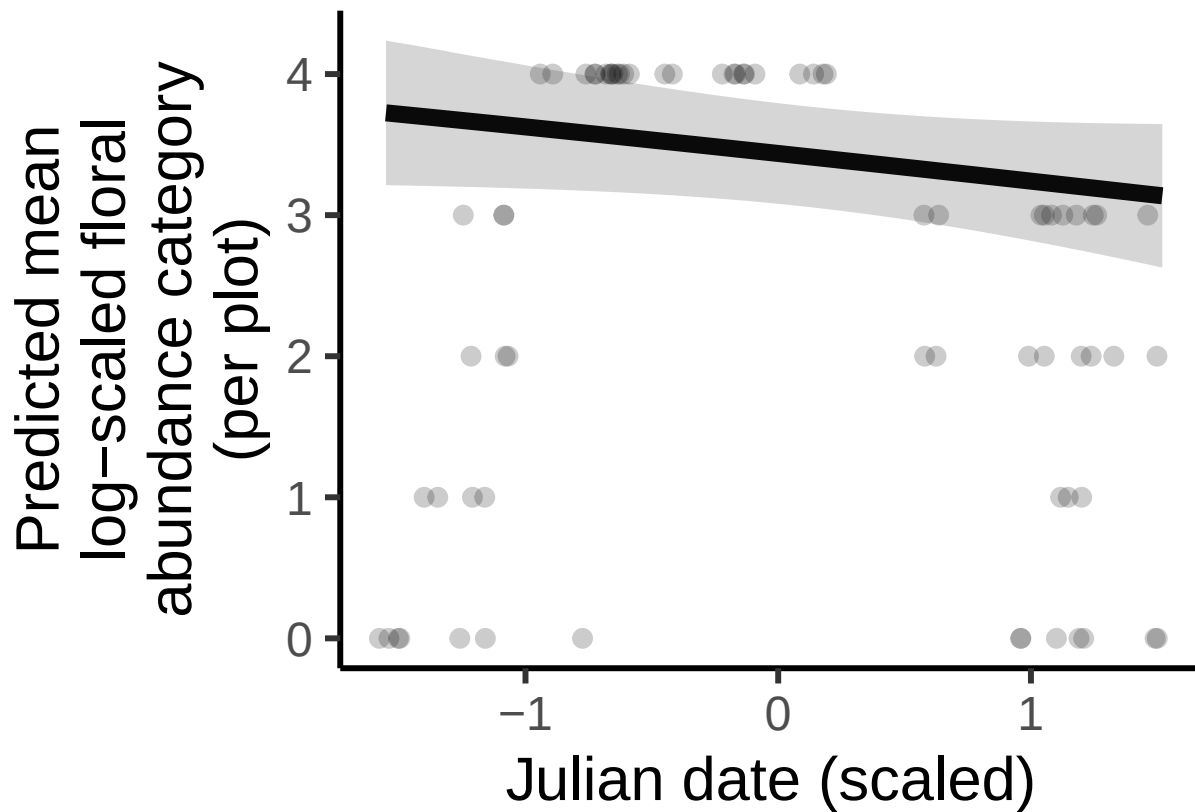
#plot

ggplot(emm_floral_date_df, aes(x = julian.date_scaled, y = mean.class)) +
  geom_jitter(
    data = subset(field_data, cover_type == "herbaceous"),
    aes(x = julian.date_scaled, y = bombus_floral_abun),
    width = 0.05,
    height = 0,
    size = 3,
    alpha = 0.2,
```

```

inherit.aes = FALSE) +
geom_line(linewidth = 3) +
geom_ribbon(
  aes(ymin = asymp.LCL, ymax = asymp.UCL),
  alpha = 0.2
) +
labs(
  x = "Julian date (scaled)",
  y = "Predicted mean\nlog-scaled floral\nabundance category\n(per plot)"
) +
theme_classic(base_size = 23)

```



Findings

No significant effect of either grass+ forb% or julian date.

Number of nest-searching queens vs flowers, rodents, and field management technique

I added bee_season_type as a covariate, since during my exploratory data analyses I saw that there was only a significant different in nest-searching queens among grassy vs bare sites during nesting season.

Steps

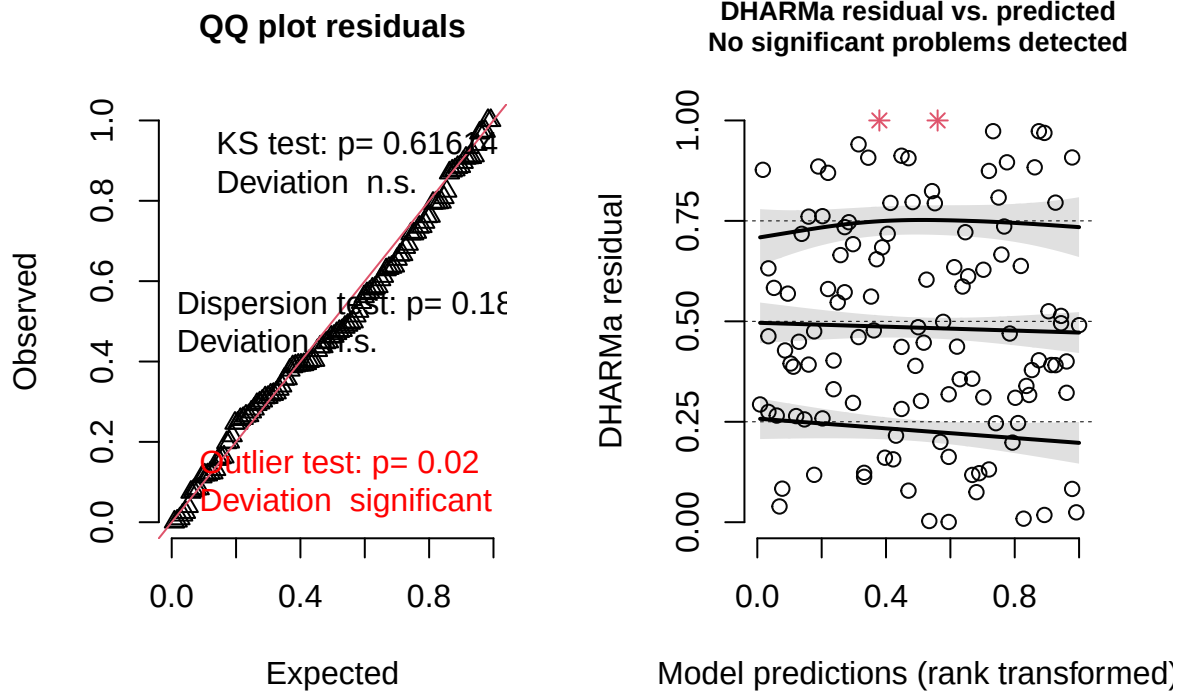
1. Run model with full random effects structure

```
NSQ_full <- glmmTMB(nest_searching_queens ~ cover_type +  
  bombus_floral_abun_scaled +  
  num_burrows_scaled +  
  prop_inactive_scaled +  
  bee_season_type +  
  (1|area/site_id/plot_id),  
  family = poisson,  
  data = field_data)
```

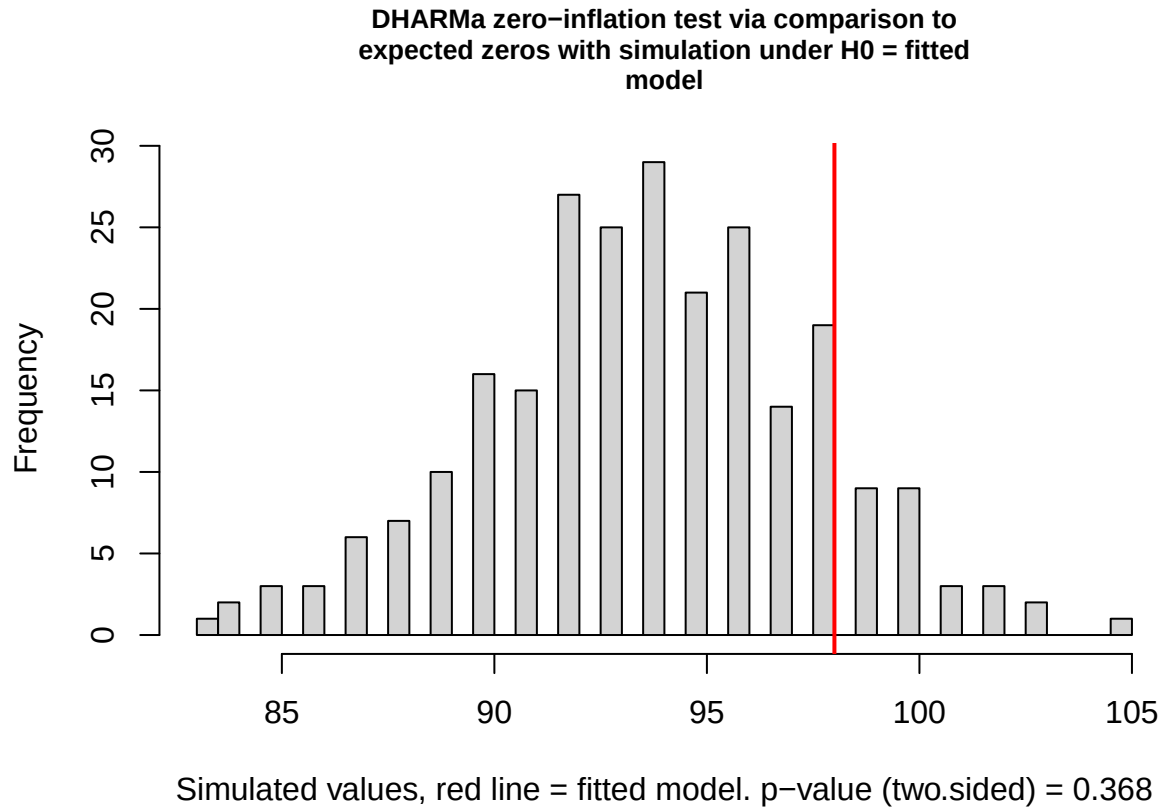
2. Check diagnostics

```
residuals_nsq_pois <- simulateResiduals(fittedModel = NSQ_full, plot = TRUE) ##significant outlier test
```

DHARMa residual



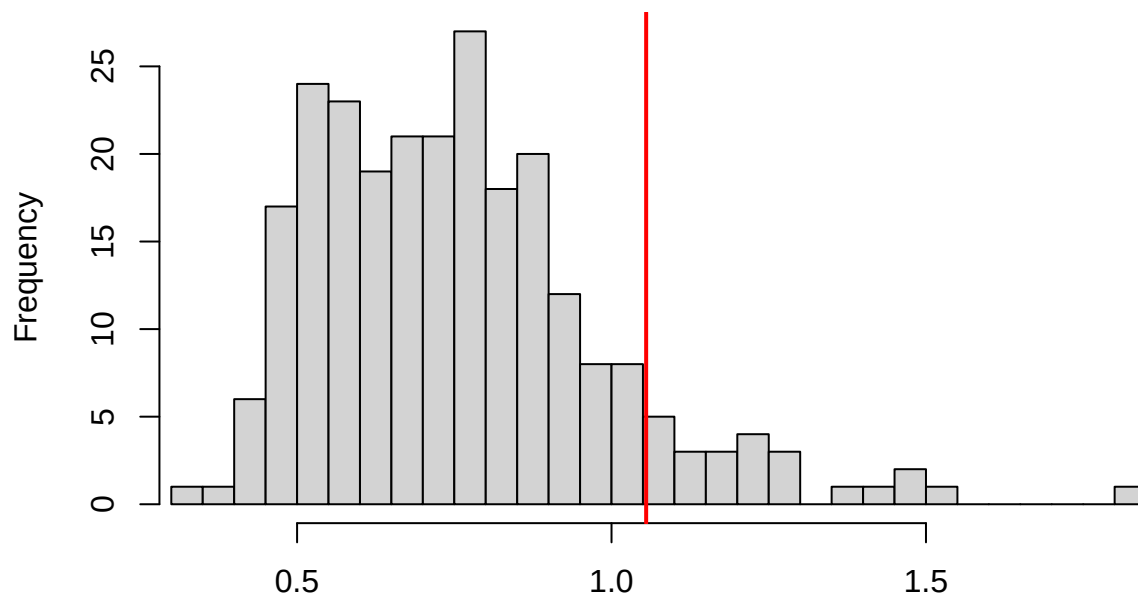
```
testZeroInflation(residuals_nsqr_pois) #no zero inflation
```



```
##  
## DHARMA zero-inflation test via comparison to expected zeros with  
## simulation under H0 = fitted model  
##  
## data: simulationOutput  
## ratioObsSim = 1.0444, p-value = 0.368  
## alternative hypothesis: two.sided
```

```
testDispersion(residuals_nsqr_pois) #no overdispersion
```

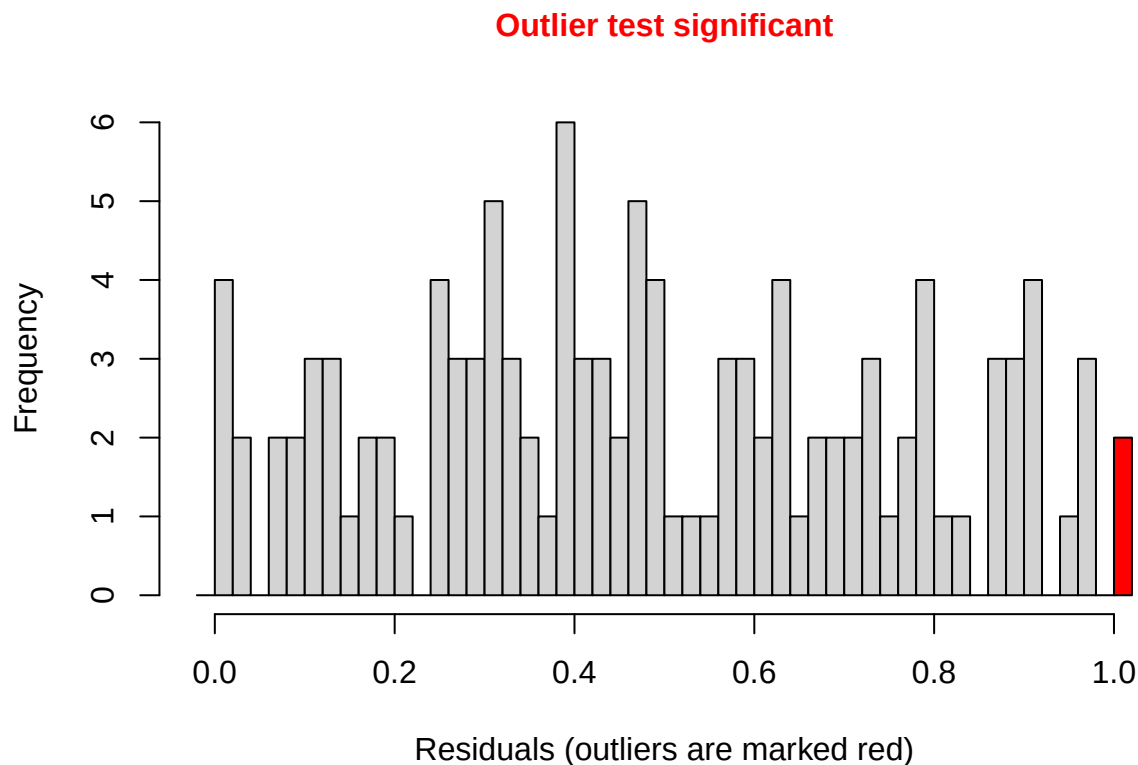
DHARMA nonparametric dispersion test via sd of residuals fitted vs. simulated



Simulated values, red line = fitted model. p-value (two.sided) = 0.184

```
##
## DHARMA nonparametric dispersion test via sd of residuals fitted vs.
## simulated
##
## data: simulationOutput
## dispersion = 1.3958, p-value = 0.184
## alternative hypothesis: two.sided
```

```
testOutliers(residuals_nsq_pois)
```



```
##
## DHARMA bootstrapped outlier test
##
## data: residuals_nsq_pois
## outliers at both margin(s) = 2, observations = 116, p-value < 2.2e-16
## alternative hypothesis: two.sided
## percent confidence interval:
## 0.00000000 0.00862069
## sample estimates:
## outlier frequency (expected: 0.00112068965517241 )
##                                0.01724138
```

4. Look at results

```
summary(NSQ_full)
```

```
## Family: poisson ( log )
## Formula:
## nest_searching_queens ~ cover_type + bombus_floral_abun_scaled +
## num_burrows_scaled + prop_inactive_scaled + bee_season_type +
## (1 | area/site_id/plot_id)
## Data: field_data
##
##      AIC      BIC    logLik -2*log(L)  df.resid
##    141.4    166.1    -61.7    123.4      107
```

```
##
## Random effects:
##
## Conditional model:
## Groups Name Variance Std.Dev.
## plot_id:site_id:area (Intercept) 9.346e-02 3.057e-01
## site_id:area (Intercept) 2.461e-09 4.961e-05
## area (Intercept) 1.036e-10 1.018e-05
## Number of obs: 116, groups:
## plot_id:site_id:area, 20; site_id:area, 11; area, 6
##
## Conditional model:
## Estimate Std. Error z value Pr(>|z|)
## (Intercept) -1.45652 0.41990 -3.469 0.000523 ***
## cover_typeherbaceous 1.01943 0.47501 2.146 0.031863 *
## bombus_floral_abun_scaled 0.25479 0.19482 1.308 0.190932
## num_burrows_scaled -0.10871 0.18026 -0.603 0.546444
## prop_inactive_scaled -0.04181 0.20337 -0.206 0.837096
## bee_season_typeworker -2.61575 0.73536 -3.557 0.000375 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Calculating predicted per burrow increase in NSQ

```
exp(1.01943) #2.8x higher in herbaceous-cover sites than bare sires
```

```
## [1] 2.771614
```

Plots

Number of nest-searching queens vs grass/forb

Conditional model:

```
#create new data grid
nsq_data_cover<- expand.grid(
  cover_type = unique(field_data$cover_type),
  bombus_floral_abun_scaled = mean(field_data$bombus_floral_abun_scaled, na.rm = TRUE),
  num_burrows_scaled = mean(field_data$num_burrows_scaled, na.rm = TRUE),
  bee_season_type = "nestsearching",
  prop_inactive_scaled = mean(field_data$prop_inactive_scaled, na.rm = TRUE)
)

# Predictions for conditional model (count)
pred_nsq_cover_cond <- predict(NSQ_full,
  newdata = nsq_data_cover,
  type = "link",
  se.fit = TRUE,
  re.form = NA)

# Combine predictions with data frame
pred_nsq_cover_df <- cbind(nsq_data_cover,
  cond_fit_log = pred_nsq_cover_cond$fit,
```



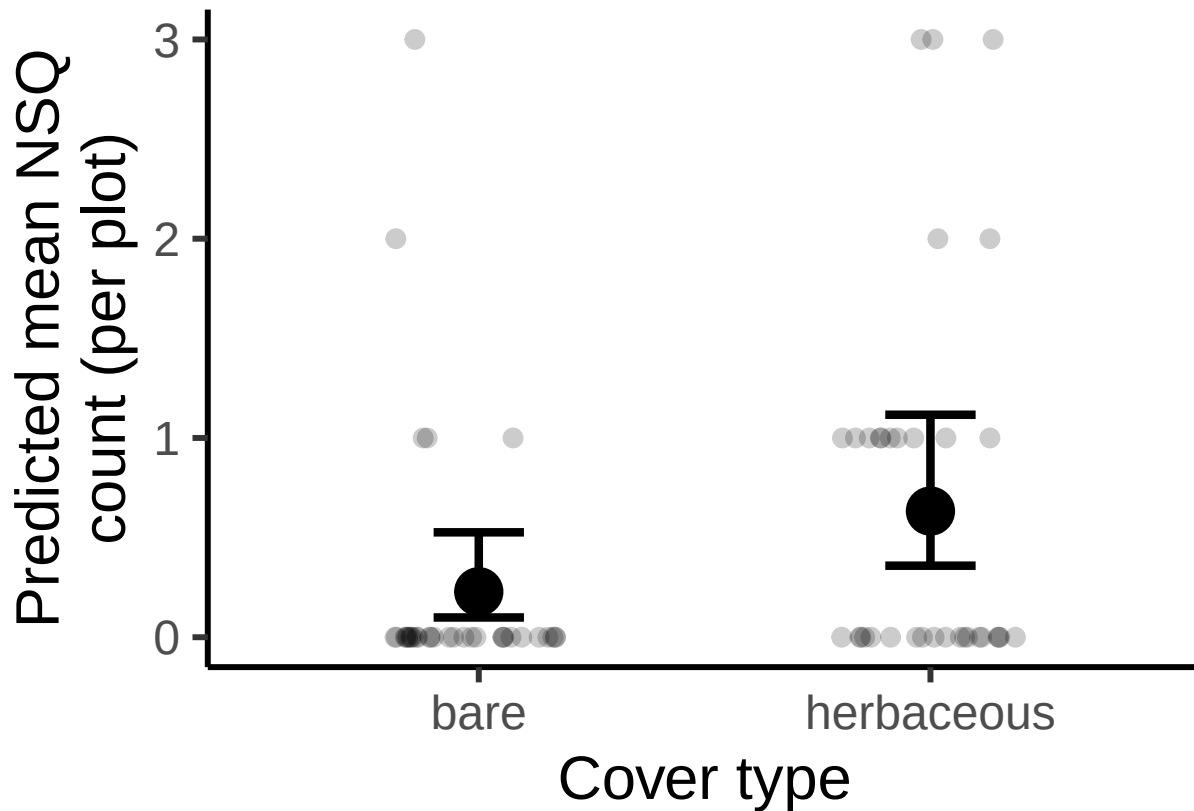
```

cond_se_log = pred_nsq_cover_cond$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_nsq_cover_df <- pred_nsq_cover_df %>%
  mutate(
    cond_fit = exp(cond_fit_log),
    cond_lower = exp(cond_fit_log - 1.96 * cond_se_log),
    cond_upper = exp(cond_fit_log + 1.96 * cond_se_log)
  )

#plot
ggplot(pred_nsq_cover_df, aes(x = cover_type, y = cond_fit)) +
  geom_jitter(
    data = subset(field_data, bee_season_type == "nestsearching"),
    aes(x = cover_type, y = nest_searching_queens),
    width = 0.2,
    height = 0,
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_point(size = 8) +
  geom_errorbar(aes(ymin = cond_lower, ymax = cond_upper), width = 0.2, position = position_dodge(0.6),
  labs(
    x = "Cover type",
    y = "Predicted mean NSQ\ncount (per plot)" ) +
  theme_classic(base_size = 23)

```



Number of nest-searching queens vs floral abundance

```
# Define a sequence of values for bombus_floral_abun_scaled across its observed range
floral_seq <- seq(
  from = min(field_data$bombus_floral_abun_scaled, na.rm = TRUE),
  to = max(field_data$bombus_floral_abun_scaled, na.rm = TRUE),
  length.out = 50
)

# Create data grid over bombus_floral_abun_scaled
nsq_data_floral <- data.frame(
  bombus_floral_abun_scaled = floral_seq,
  cover_type = "herbaceous",
  num_burrows_scaled = mean(field_data$num_burrows_scaled, na.rm = TRUE),
  prop_inactive_scaled = mean(field_data$prop_inactive_scaled, na.rm = TRUE),
  bee_season_type = "nestsearching"
)

# Predictions for conditional model (count)
pred_nsq_floral_cond <- predict(NSQ_full,
  newdata = nsq_data_floral,
  type = "link",
  se.fit = TRUE,
  re.form = NA)

# Combine predictions with newdata
```

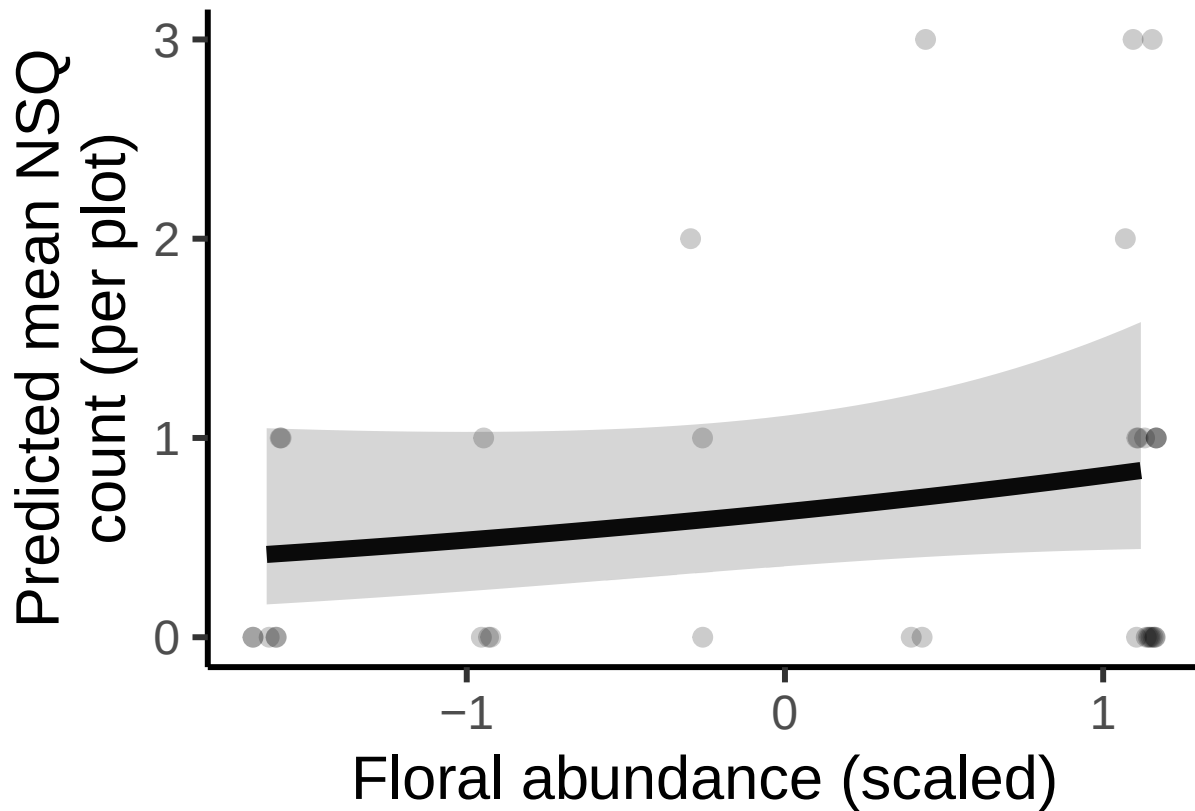
```

pred_nsq_floral_df <- cbind(nsq_data_floral,
                           cond_fit_log = pred_nsq_floral_cond$fit,
                           cond_se_log = pred_nsq_floral_cond$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_nsq_floral_df <- pred_nsq_floral_df %>%
  mutate(
    cond_fit = exp(cond_fit_log),
    cond_lower = exp(cond_fit_log - 1.96 * cond_se_log),
    cond_upper = exp(cond_fit_log + 1.96 * cond_se_log)
  )

# Plot conditional predictions
ggplot(pred_nsq_floral_df, aes(x = bombus_floral_abun_scaled, y = cond_fit)) +
  geom_jitter(
    data = subset(field_data, bee_season_type == "nestsearching" & cover_type == "herbaceous"),
    aes(x = bombus_floral_abun_scaled, y = nest_searching_queens),
    width = 0.05,
    height = 0,
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_line(linewidth = 3) +
  geom_ribbon(aes(ymin = cond_lower, ymax = cond_upper), alpha = 0.2) +
  labs(x = "Floral abundance (scaled)", y = "Predicted mean NSQ\ncount (per plot)") +
  theme_classic(base_size = 23)

```



Number of nest-searching queens vs burrow vacancy proportion

```
#create new data grid
nsq_data_vacancy<- expand.grid(
  cover_type = "herbaceous",
  bombus_floral_abun_scaled = mean(field_data$bombus_floral_abun_scaled, na.rm = TRUE),
  num_burrows_scaled = mean(field_data$num_burrows_scaled, na.rm = TRUE),
  bee_season_type = "nestsearching",
  prop_inactive_scaled = seq(min(field_data$prop_inactive_scaled, na.rm = TRUE),
                             max(field_data$prop_inactive_scaled, na.rm = TRUE),
                             length.out = 50)
)

# Predictions for conditional model (count)
pred_nsq_vacancy_cond <- predict(NSQ_full,
                                newdata = nsq_data_vacancy,
                                type = "link",
                                se.fit = TRUE,
                                re.form = NA)

# Combine predictions with data frame
pred_nsq_vacancy_df <- cbind(nsq_data_vacancy,
                             cond_fit_log = pred_nsq_vacancy_cond$fit,
                             cond_se_log = pred_nsq_vacancy_cond$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
```

```

pred_nsq_vacancy_df <- pred_nsq_vacancy_df %>%
  mutate(
    cond_fit = exp(cond_fit_log),
    cond_lower = exp(cond_fit_log - 1.96 * cond_se_log),
    cond_upper = exp(cond_fit_log + 1.96 * cond_se_log)
  )

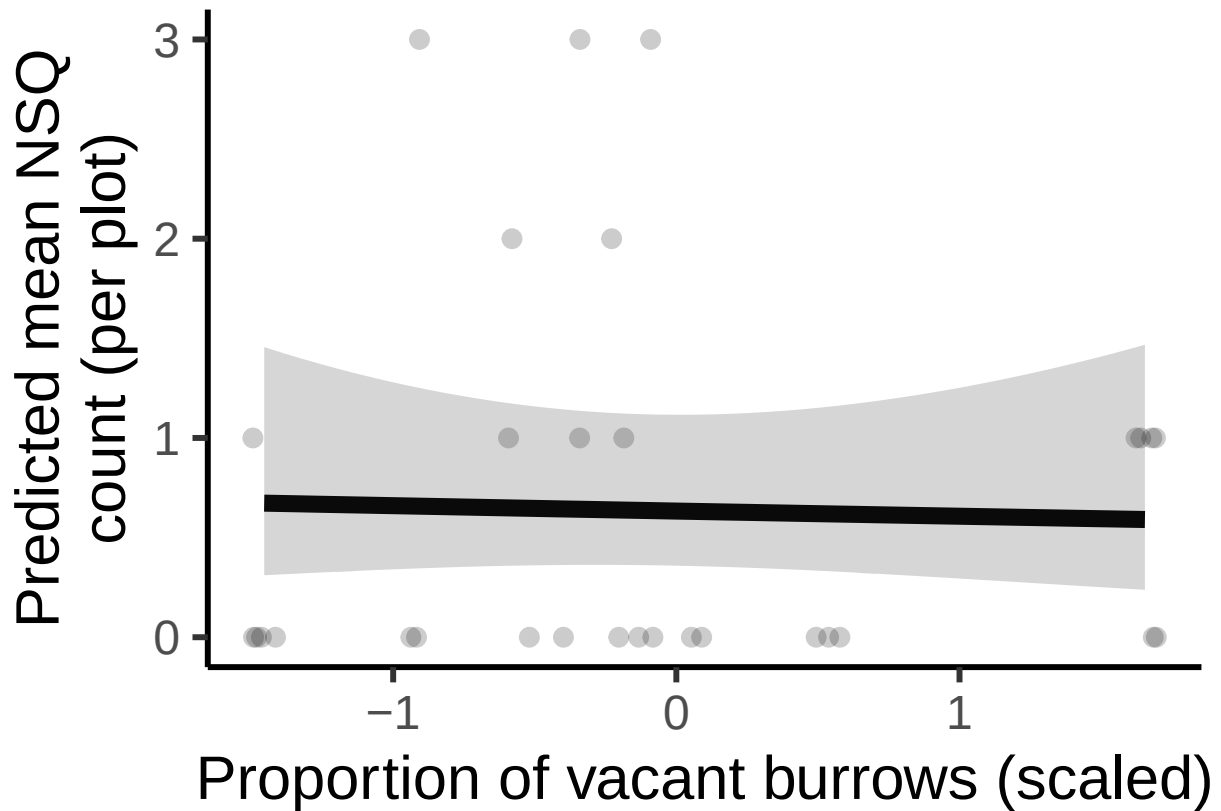
#plot
ggplot(pred_nsq_vacancy_df, aes(x = prop_inactive_scaled, y = cond_fit)) +
  geom_jitter(
    data = subset(field_data, bee_season_type == "nestsearching" & cover_type == "herbaceous"),
    aes(x = prop_inactive_scaled, y = nest_searching_queens),
    width = 0.05,
    height = 0,
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_line(linewidth = 3) +
  geom_ribbon(aes(ymin = cond_lower, ymax = cond_upper), alpha = 0.2) +
  labs(
    x = "Proportion of vacant burrows (scaled)",
    y = "Predicted mean NSQ\ncount (per plot)") +
  theme_classic(base_size = 23)

```

```

## Warning: Removed 3 rows containing missing values or values outside the scale range
## ('geom_point()').

```



Number of nest-searching queens vs number of rodent burrows

```
# Create data grid over rodent_burrows_scaled
nsq_data_rodent <- data.frame(
  num_burrows_scaled = seq(min(field_data$num_burrows_scaled), max(field_data$num_burrows_scaled), length.out = 100),
  prop_inactive_scaled = mean(field_data$prop_inactive_scaled, na.rm = TRUE),
  cover_type = "herbaceous",
  bombus_floral_abun_scaled = mean(field_data$bombus_floral_abun_scaled, na.rm = TRUE),
  bee_season_type = "nestsearching"
)

# Predictions for conditional model (count)
pred_nsq_rodent_cond <- predict(
  NSQ_full,
  newdata = nsq_data_rodent,
  type = "link", ##log-scale
  se.fit = TRUE,
  re.form = NA)

# Combine predictions with newdata
pred_nsq_rodent_df <- cbind(nsq_data_rodent,
  cond_fit_log = pred_nsq_rodent_cond$fit,
  cond_se_log = pred_nsq_rodent_cond$se.fit)

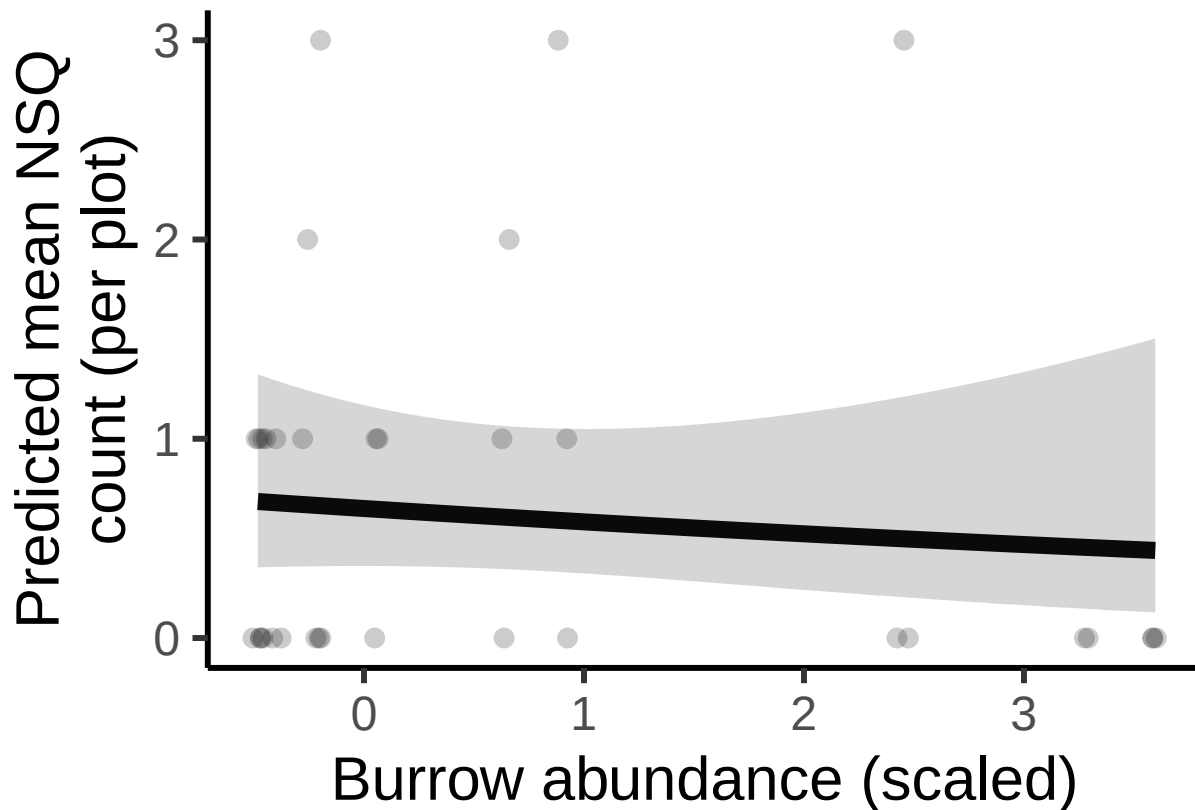
#calculate predictions and confidence intervals on response scale (not log scale)
pred_nsq_rodent_df <- pred_nsq_rodent_df %>%
```

```

mutate(
  cond_fit = exp(cond_fit_log),
  cond_lower = exp(cond_fit_log - 1.96 * cond_se_log),
  cond_upper = exp(cond_fit_log + 1.96 * cond_se_log)
)

ggplot(pred_nsq_rodent_df, aes(x = num_burrows_scaled, y = cond_fit)) +
  geom_jitter(
    data = subset(field_data, bee_season_type == "nestsearching" & cover_type == "herbaceous"),
    aes(x = num_burrows_scaled, y = nest_searching_queens),
    width = 0.05,
    height = 0,
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_line(linewidth = 3) +
  geom_ribbon(aes(ymin = cond_lower, ymax = cond_upper), alpha = 0.2) +
  labs(
    x = "Burrow abundance (scaled)",
    y = "Predicted mean NSQ\ncount (per plot)"
  ) +
  theme_classic(base_size = 23)

```



Number of nest-searching queens vs season type

```

# Create newdata with discrete factor levels for bee_season_type
nsq_data_season <- data.frame(
  cover_type = "herbaceous",
  bombus_floral_abun_scaled = mean(field_data$bombus_floral_abun_scaled),
  num_burrows_scaled = mean(field_data$num_burrows_scaled, na.rm = TRUE),
  prop_inactive_scaled = mean(field_data$prop_inactive_scaled, na.rm = TRUE),
  bee_season_type = factor(c("nestsearching", "worker"), levels = levels(field_data$bee_season_type))
)

# Predictions for conditional model (count)
pred_nsq_season_cond <- predict(
  NSQ_full,
  newdata = nsq_data_season,
  type = "link", ##log-scale
  se.fit = TRUE,
  re.form = NA)

# Combine predictions with newdata
pred_nsq_season_df <- cbind(nsq_data_season,
  cond_fit_log = pred_nsq_season_cond$fit,
  cond_se_log = pred_nsq_season_cond$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_nsq_season_df <- pred_nsq_season_df %>%
  mutate(
    cond_fit = exp(cond_fit_log),
    cond_lower = exp(cond_fit_log - 1.96 * cond_se_log),
    cond_upper = exp(cond_fit_log + 1.96 * cond_se_log)
  )

# Plot conditional predictions
ggplot(pred_nsq_season_df, aes(x = bee_season_type, y = cond_fit)) +
  geom_jitter(
    data = subset(field_data, cover_type == "herbaceous"),
    aes(x = bee_season_type, y = nest_searching_queens),
    width = 0.2,
    height = 0,
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_point(size=8) +
  geom_errorbar(aes(ymin = cond_lower, ymax = cond_upper), width = 0.2, position = position_dodge(0.6),
  labs(x = "Bee season", y = "Predicted mean NSQ\ncount (per plot)") +
  theme_classic(base_size = 23)

```


Number of workers vs nest-searching queens, floral abundance, rodent burrows.

Go through similar model selection steps as for the rodent burrow model.

Steps

- 1) Run full model

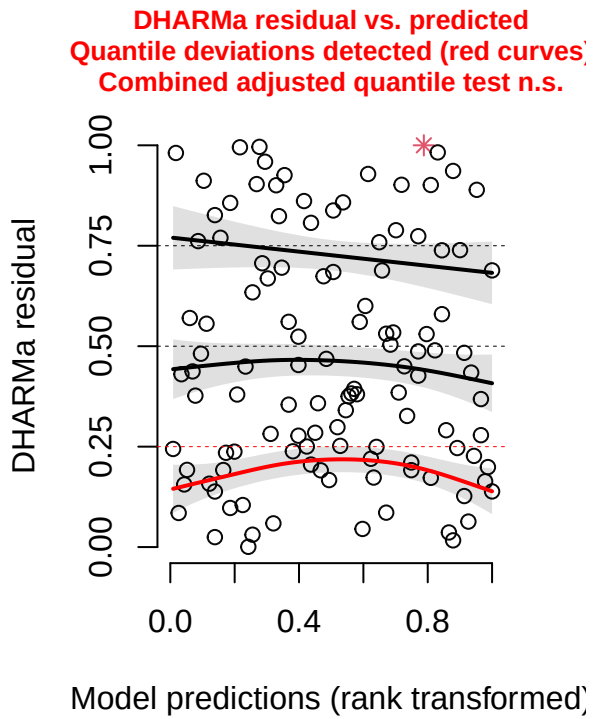
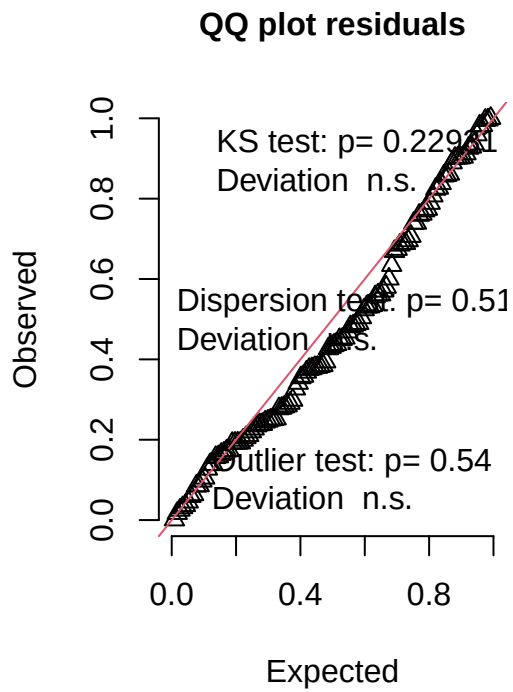
```
workers_full <- glmmTMB(workers ~ nest_searching_queens_scaled +  
  cover_type +  
  bombus_floral_abun_scaled +  
  num_burrows_scaled +  
  prop_inactive_scaled +  
  bee_season_type +  
  (1|area/site_id/plot_id),  
  family = poisson,  
  data = field_data  
)
```

2. Check diagnostics of model.

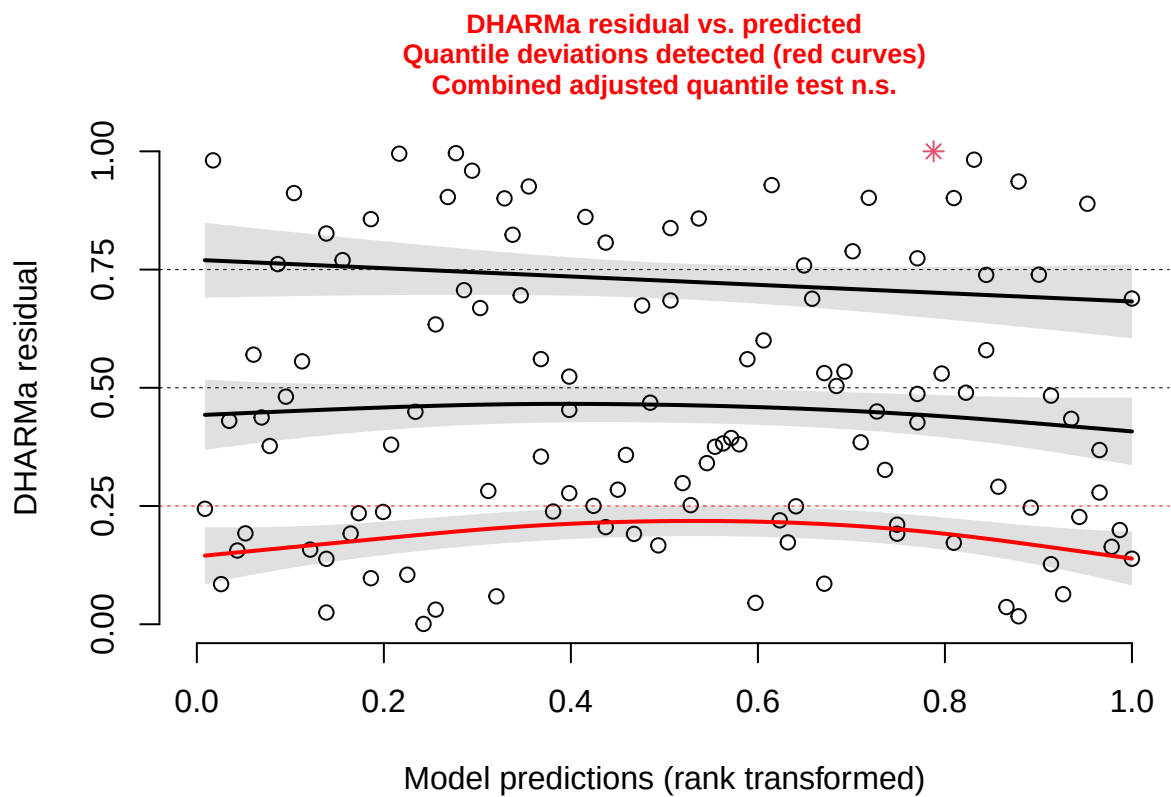
- Combined quantile test significant. No

```
#simulate residuals  
residuals_workers <- simulateResiduals(fittedModel = workers_full, plot = TRUE)
```

DHARMa residual



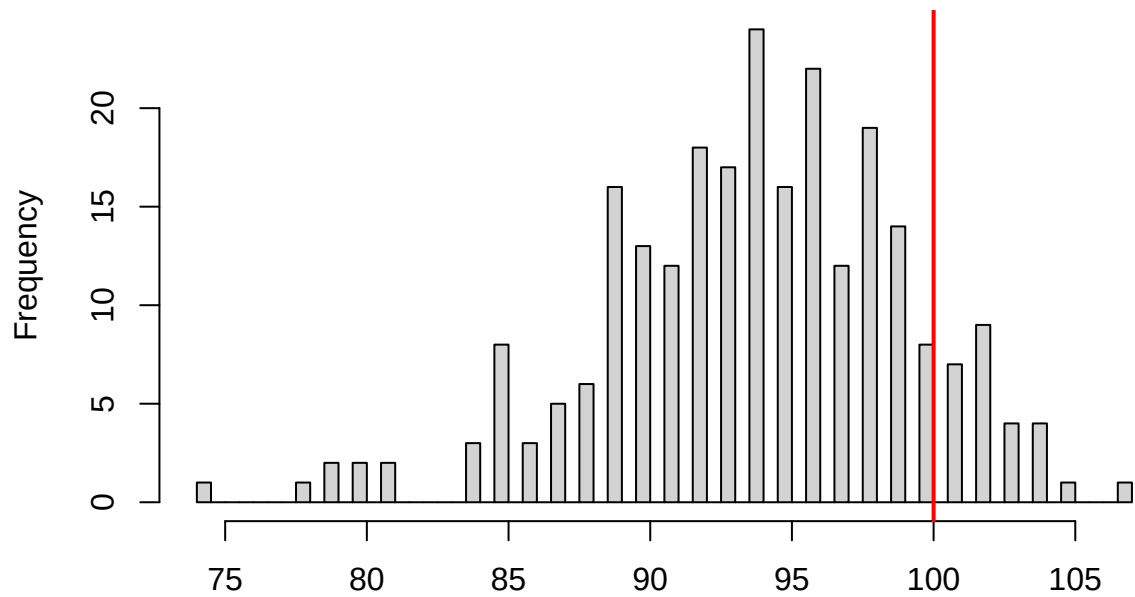
```
testQuantiles(residuals_workers)
```



```
##
## Test for location of quantiles via qgam
##
## data: res
## p-value = 0.05234
## alternative hypothesis: both
```

```
testZeroInflation(residuals_workers) #no zero inflation
```

DHARMA zero-inflation test via comparison to expected zeros with simulation under H0 = fitted model

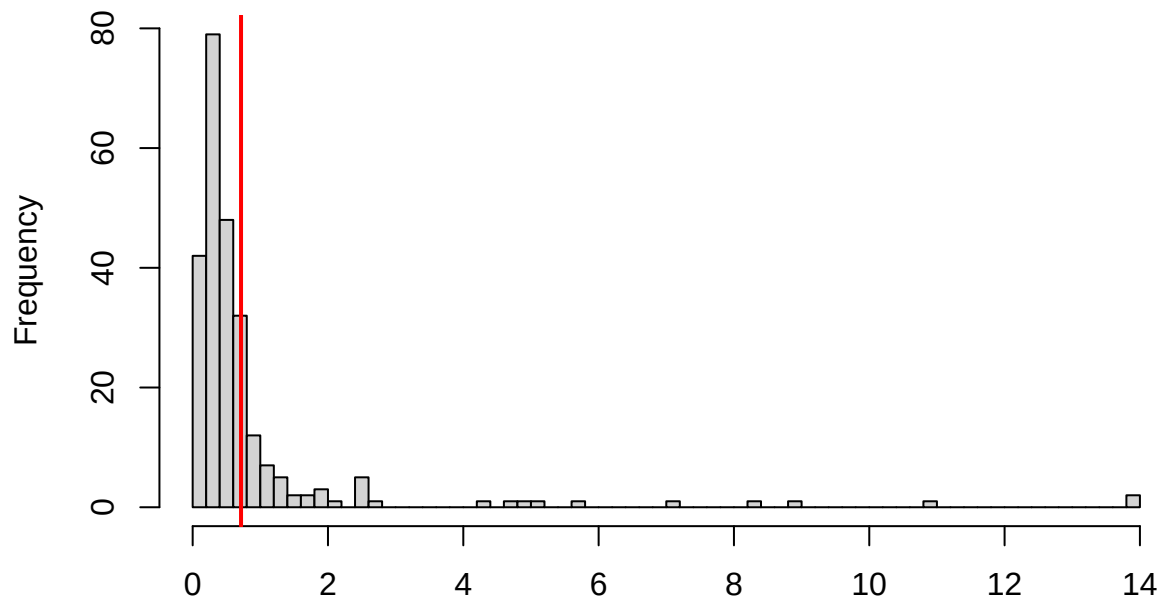


Simulated values, red line = fitted model. p-value (two.sided) = 0.272

```
##
## DHARMA zero-inflation test via comparison to expected zeros with
## simulation under H0 = fitted model
##
## data: simulationOutput
## ratioObsSim = 1.0651, p-value = 0.272
## alternative hypothesis: two.sided
```

```
testDispersion(residuals_workers) #no overdispersion
```

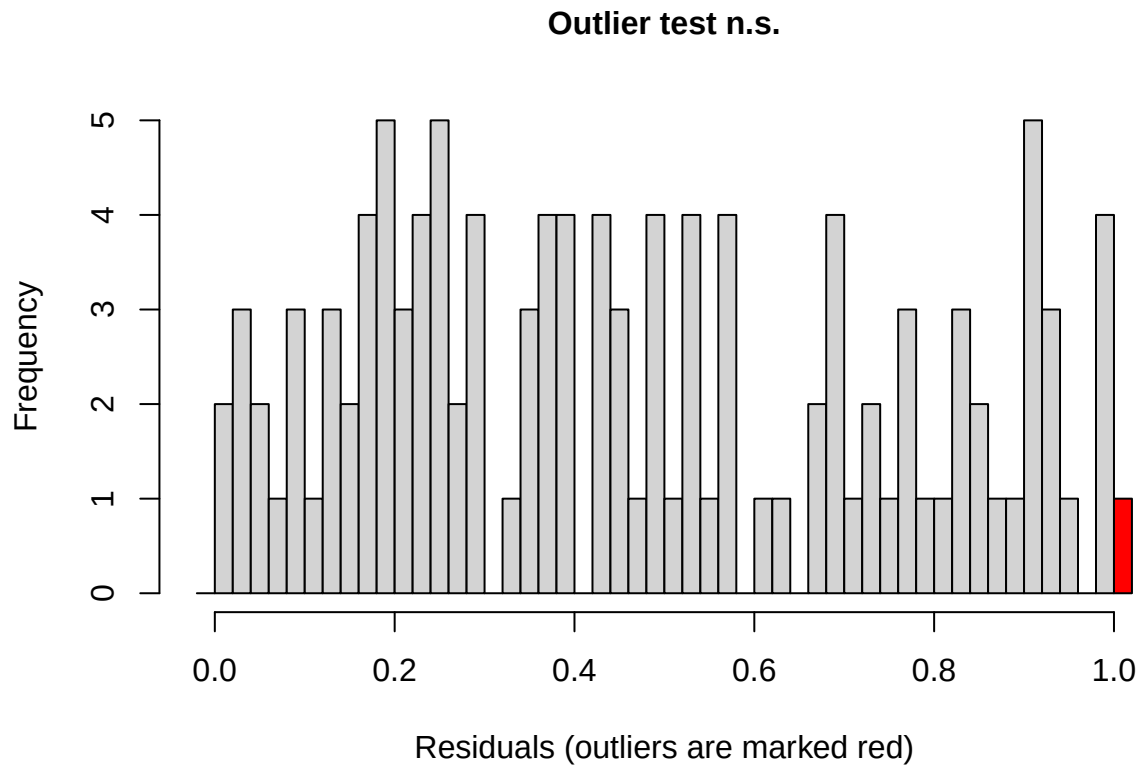
**DHARMA nonparametric dispersion test via sd of
residuals fitted vs. simulated**



Simulated values, red line = fitted model. p-value (two.sided) = 0.512

```
##  
## DHARMA nonparametric dispersion test via sd of residuals fitted vs.  
## simulated  
##  
## data: simulationOutput  
## dispersion = 0.82587, p-value = 0.512  
## alternative hypothesis: two.sided
```

```
testOutliers(residuals_workers) #ns
```



```
##
## DHARMA bootstrapped outlier test
##
## data: residuals_workers
## outliers at both margin(s) = 1, observations = 116, p-value = 0.5
## alternative hypothesis: two.sided
## percent confidence interval:
## 0.00000000 0.02586207
## sample estimates:
## outlier frequency (expected: 0.00353448275862069 )
##                                0.00862069
```

3. Check results of final model

```
summary(workers_full)
```

```
## Family: poisson ( log )
## Formula:
## workers ~ nest_searching_queens_scaled + cover_type + bombus_floral_abun_scaled +
## num_burrows_scaled + prop_inactive_scaled + bee_season_type +
## (1 | area/site_id/plot_id)
## Data: field_data
##
##      AIC      BIC    logLik -2*log(L)  df.resid
##    168.2    195.7    -74.1    148.2      106
```

```
##
## Random effects:
##
## Conditional model:
## Groups Name Variance Std.Dev.
## plot_id:site_id:area (Intercept) 5.458e-01 7.388e-01
## site_id:area (Intercept) 5.324e-01 7.297e-01
## area (Intercept) 4.873e-09 6.981e-05
## Number of obs: 116, groups:
## plot_id:site_id:area, 20; site_id:area, 11; area, 6
##
## Conditional model:
## Estimate Std. Error z value Pr(>|z|)
## (Intercept) -3.38303 0.75505 -4.481 7.44e-06 ***
## nest_searching_queens_scaled 0.07049 0.55542 0.127 0.89901
## cover_typeherbaceous -0.69886 0.86497 -0.808 0.41912
## bombus_floral_abun_scaled 0.72505 0.22313 3.249 0.00116 **
## num_burrows_scaled 0.37962 0.31973 1.187 0.23510
## prop_inactive_scaled -0.26308 0.37674 -0.698 0.48498
## bee_season_typerworker 2.36906 0.61924 3.826 0.00013 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Calculate estimate of floral abundance on response scale

```
sd_floral <- sd(field_data$bombus_floral_abun)
exp(0.72505 / sd_floral) ##1.61
```

```
## [1] 1.608099
```

Plots

Number of workers vs number of nest-searching queens

```
# Define a sequence of values for nsq across its observed range
queen_seq <- seq(
  from = min(field_data$nest_searching_queens_scaled, na.rm = TRUE),
  to = max(field_data$nest_searching_queens_scaled, na.rm = TRUE),
  length.out = 50
)

#create new data grid
worker_data_nsq<- expand.grid(
  nest_searching_queens_scaled = queen_seq,
  cover_type = "herbaceous",
  bombus_floral_abun_scaled = mean(field_data$bombus_floral_abun_scaled, na.rm = TRUE),
  num_burrows_scaled = mean(field_data$num_burrows_scaled, na.rm = TRUE),
  prop_inactive_scaled = mean(field_data$prop_inactive_scaled, na.rm = TRUE),
  bee_season_type = "worker"
)

# Get predicted probabilities and standard errors
```



```

pred_worker_nsqr <- predict(workers_full,
                             newdata = worker_data_nsqr,
                             type = "link",
                             se.fit = TRUE,
                             re.form = NA)

# Combine predictions with data frame
pred_worker_nsqr_df <- cbind(worker_data_nsqr,
                              fit_log = pred_worker_nsqr$fit,
                              se_log = pred_worker_nsqr$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_worker_nsqr_df <- pred_worker_nsqr_df %>%
  mutate(
    fit = exp(fit_log),
    lower = exp(fit_log - 1.96 * se_log),
    upper = exp(fit_log + 1.96 * se_log)
  )

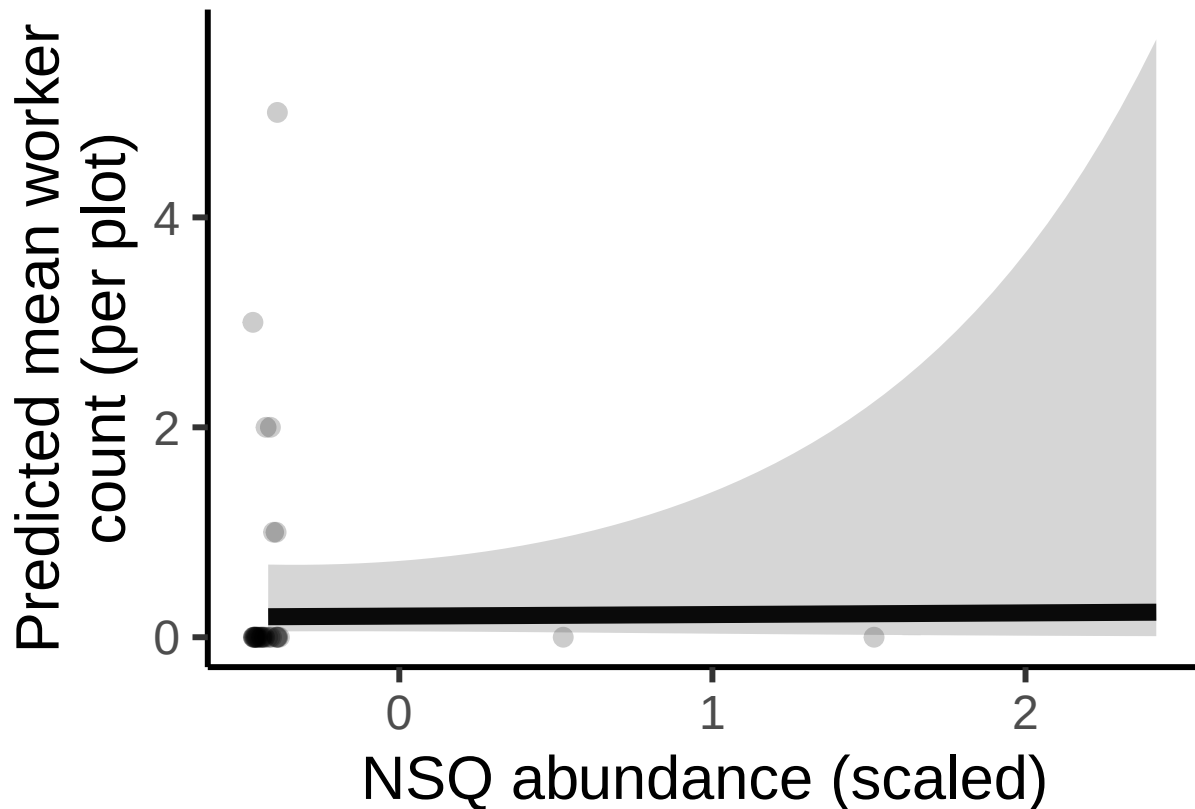
ggplot(pred_worker_nsqr_df, aes(x = nest_searching_queens_scaled, y = fit)) +
  geom_jitter(
    data = subset(field_data, bee_season_type == "worker" & cover_type == "herbaceous"),
    aes(x = nest_searching_queens_scaled, y = workers),
    width = 0.05,
    height = 0,
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_line(size = 3) +
  geom_ribbon(aes(ymin = lower, ymax = upper), alpha = 0.2) +
  labs(
    x = "NSQ abundance (scaled)",
    y = "Predicted mean worker\ncount (per plot)" +
  theme_classic(base_size = 23)

```

```

## Warning: Using 'size' aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use 'linewidth' instead.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.

```



Number of workers vs floral abundance

```
#create new data grid
worker_data_floral<- expand.grid(
  nest_searching_queens_scaled = mean(field_data$nest_searching_queens_scaled, na.rm = TRUE),
  cover_type = "herbaceous",
  bombus_floral_abun_scaled = floral_seq,
  num_burrows_scaled = mean(field_data$num_burrows_scaled, na.rm = TRUE),
  prop_inactive_scaled = mean(field_data$prop_inactive_scaled, na.rm = TRUE),
  bee_season_type = "worker"
)

# Get predicted probabilities and standard errors
pred_worker_floral <- predict(workers_full,
                             newdata = worker_data_floral,
                             type = "link",
                             se.fit = TRUE,
                             re.form = NA)

# Combine predictions with data frame
pred_worker_floral_df <- cbind(worker_data_floral,
                               fit_log = pred_worker_floral$fit,
                               se_log = pred_worker_floral$se.fit)

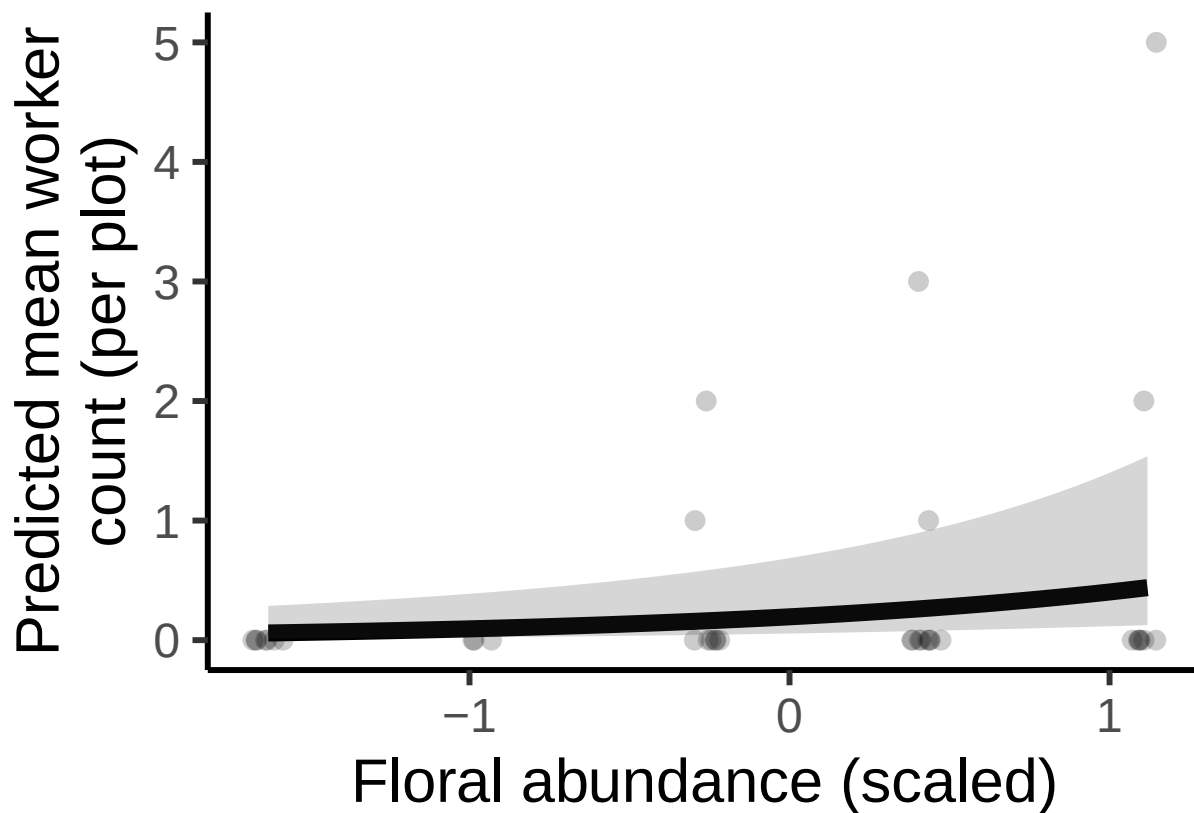
#calculate predictions and confidence intervals on response scale (not log scale)
pred_worker_floral_df <- pred_worker_floral_df %>%
```

```

mutate(
  fit = exp(fit_log),
  lower = exp(fit_log - 1.96 * se_log),
  upper = exp(fit_log + 1.96 * se_log)
)

ggplot(pred_worker_floral_df, aes(x = bombus_floral_abun_scaled, y = fit)) +
  geom_jitter(
    data = subset(field_data, bee_season_type == "worker" & cover_type == "herbaceous"),
    aes(x = bombus_floral_abun_scaled, y = workers),
    width = 0.05,
    height = 0,
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_line(size = 3) +
  geom_ribbon(aes(ymin = lower, ymax = upper), alpha = 0.2) +
  labs(
    x = "Floral abundance (scaled)",
    y = "Predicted mean worker\ncount (per plot)" +
  theme_classic(base_size = 23)

```



Number of workers vs number of rodent burrows

```

# Define a sequence of values for rodent burrows across its observed range
rodent_seq <- seq(
  from = min(field_data$num_burrows_scaled, na.rm = TRUE),
  to = max(field_data$num_burrows_scaled, na.rm = TRUE),
  length.out = 50)

#create new data grid
worker_data_rodent<- expand.grid(
  nest_searching_queens_scaled = mean(field_data$nest_searching_queens_scaled, na.rm = TRUE),
  cover_type = "herbaceous",
  bombus_floral_abun_scaled = mean(field_data$bombus_floral_abun_scaled, na.rm = TRUE),
  num_burrows_scaled = rodent_seq,
  prop_inactive_scaled = mean(field_data$prop_inactive_scaled, na.rm = TRUE),
  bee_season_type = "worker"
)

# Get predicted probabilities and standard errors
pred_worker_rodent <- predict(workers_full,
                             newdata = worker_data_rodent,
                             type = "link",
                             se.fit = TRUE,
                             re.form = NA)

# Combine predictions with data frame
pred_worker_rodent_df <- cbind(worker_data_rodent,
                               fit_log = pred_worker_rodent$fit,
                               se_log = pred_worker_rodent$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_worker_rodent_df <- pred_worker_rodent_df %>%
  mutate(
    fit = exp(fit_log),
    lower = exp(fit_log - 1.96 * se_log),
    upper = exp(fit_log + 1.96 * se_log)
  )

ggplot(pred_worker_rodent_df, aes(x = num_burrows_scaled, y = fit)) +
  geom_jitter(
    data = subset(field_data, bee_season_type == "worker" & cover_type == "herbaceous"),
    aes(x = num_burrows_scaled, y = workers),
    width = 0.05,
    height = 0,
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_line(size = 3) +
  geom_ribbon(aes(ymin = lower, ymax = upper), alpha = 0.2) +
  labs(
    x = "Burrow abundance (scaled)",
    y = "Predicted mean worker\ncount (per plot)") +
  theme_classic(base_size = 23)

```



Number of workers vs burrow vacancy proportion

```
#create new data grid
worker_data_vacancy <- expand.grid(
  nest_searching_queens_scaled = mean(field_data$nest_searching_queens_scaled, na.rm = TRUE),
  cover_type = "herbaceous",
  bombus_floral_abun_scaled = mean(field_data$bombus_floral_abun_scaled, na.rm = TRUE),
  num_burrows_scaled = mean(field_data$num_burrows_scaled, na.rm = TRUE),
  bee_season_type = "worker",
  prop_inactive_scaled = seq(min(field_data$prop_inactive_scaled, na.rm = TRUE),
                             max(field_data$prop_inactive_scaled, na.rm = TRUE),
                             length.out = 50)
)

# Predictions
pred_worker_vacancy <- predict(workers_full,
                              newdata = worker_data_vacancy,
                              type = "link",
                              se.fit = TRUE,
                              re.form = NA
)

# Combine predictions with data frame
pred_worker_vacancy_df <- cbind(worker_data_vacancy,
                                fit_log = pred_worker_vacancy$fit,
                                se_log = pred_worker_vacancy$se.fit)
```

```

#calculate predictions and confidence intervals on response scale (not log scale)
pred_worker_vacancy_df <- pred_worker_vacancy_df %>%
  mutate(
    fit = exp(fit_log),
    lower = exp(fit_log - 1.96 * se_log),
    upper = exp(fit_log + 1.96 * se_log)
  )

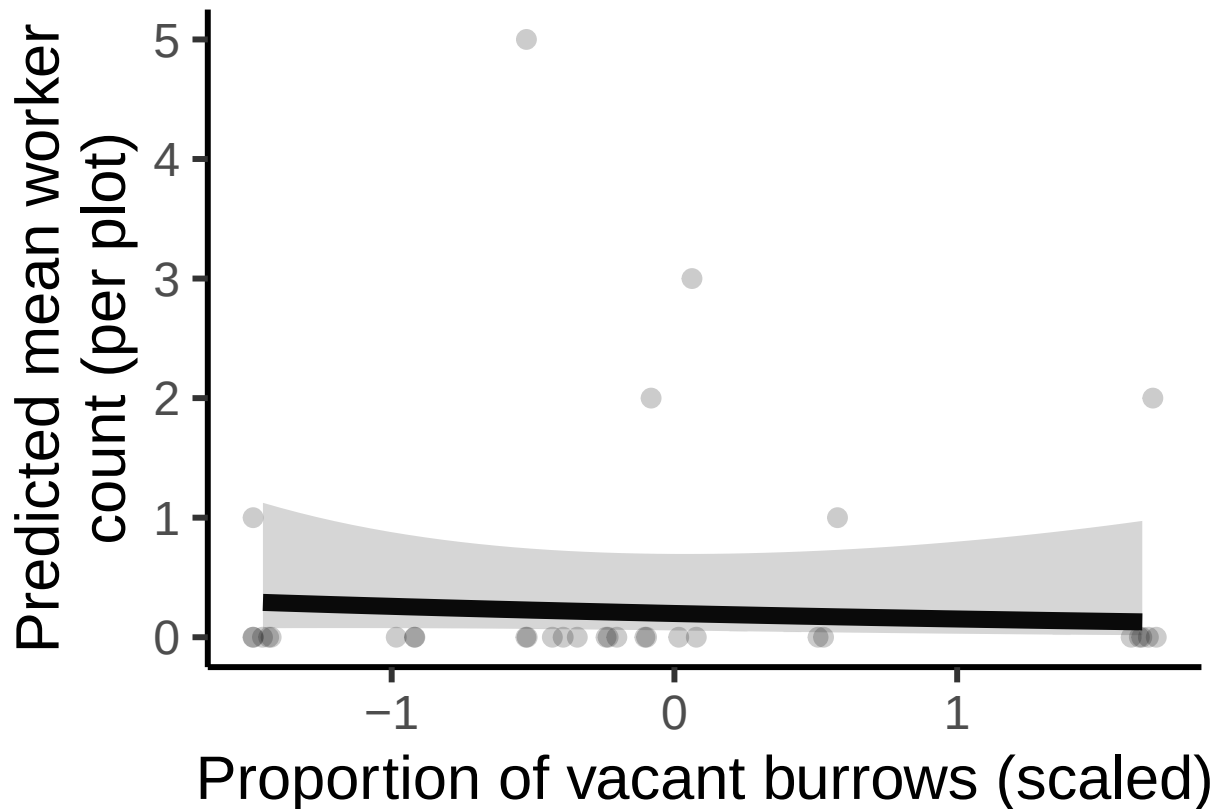
#plot
ggplot(pred_worker_vacancy_df, aes(x = prop_inactive_scaled, y = fit)) +
  geom_jitter(
    data = subset(field_data, bee_season_type == "worker" & cover_type == "herbaceous"),
    aes(x = prop_inactive_scaled, y = workers),
    width = 0.05,
    height = 0,
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_line(linewidth = 3) +
  geom_ribbon(aes(ymin = lower, ymax = upper), alpha = 0.2) +
  labs(
    x = "Proportion of vacant burrows (scaled)",
    y = "Predicted mean worker\ncount (per plot)") +
  theme_classic(base_size = 23)

```

```

## Warning: Removed 3 rows containing missing values or values outside the scale range
## ('geom_point()').

```



Number of workers vs cover type

```
#create new data grid
worker_data_season<- expand.grid(
  nest_searching_queens_scaled = mean(field_data$nest_searching_queens_scaled, na.rm = TRUE),
  bombus_floral_abun_scaled = mean(field_data$bombus_floral_abun_scaled, na.rm = TRUE),
  num_burrows_scaled = mean(field_data$num_burrows_scaled, na.rm = TRUE),
  prop_inactive_scaled = mean(field_data$prop_inactive_scaled, na.rm = TRUE),
  bee_season_type = "worker",
  cover_type = unique(field_data$cover_type)
)

# Get predicted probabilities and standard errors
pred_worker_season <- predict(workers_full,
  newdata = worker_data_season,
  type = "link",
  se.fit = TRUE,
  re.form = NA)

# Combine predictions with data frame
pred_worker_season_df <- cbind(worker_data_season,
  fit_log = pred_worker_season$fit,
  se_log = pred_worker_season$se.fit)

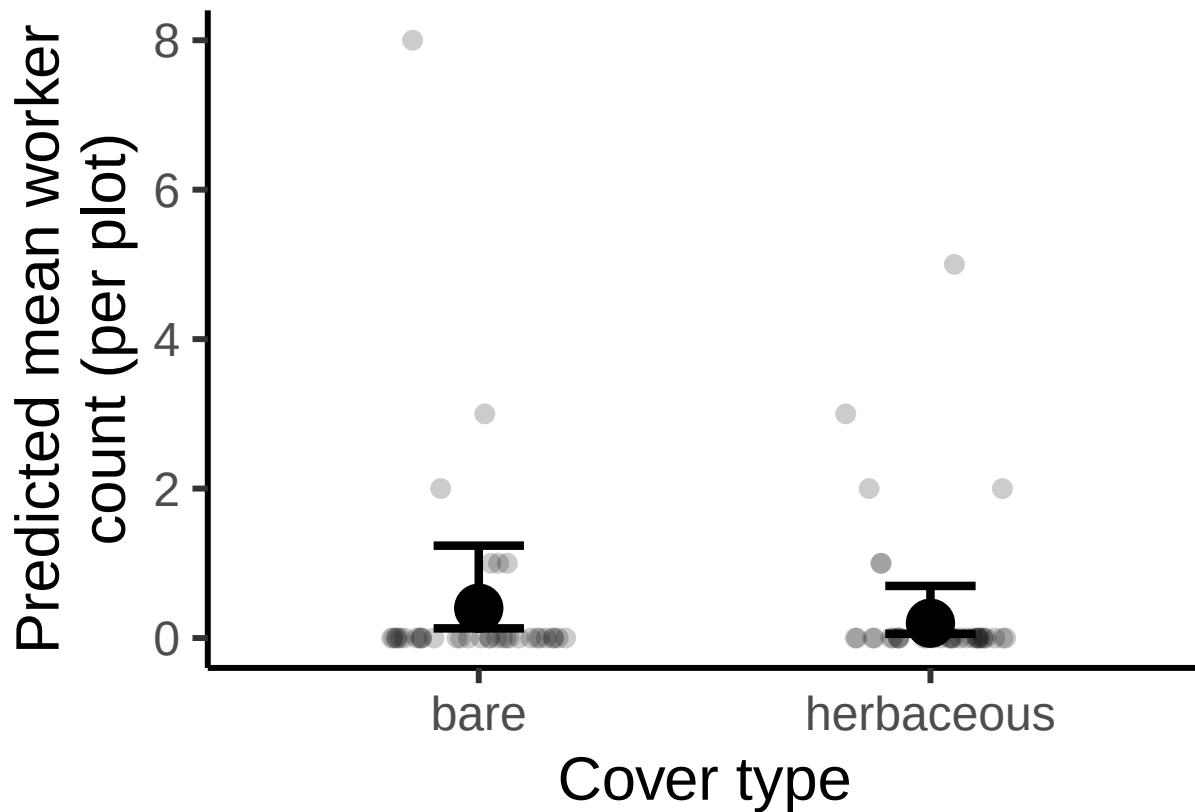
#calculate predictions and confidence intervals on response scale (not log scale)
pred_worker_season_df <- pred_worker_season_df %>%
```

```

mutate(
  fit = exp(fit_log),
  lower = exp(fit_log - 1.96 * se_log),
  upper = exp(fit_log + 1.96 * se_log)
)

ggplot(pred_worker_season_df, aes(x = cover_type, y = fit)) +
  geom_jitter(
    data = subset(field_data, bee_season_type == "worker"),
    aes(x = cover_type, y = workers),
    width = 0.2,
    height = 0,
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_point(size = 8) +
  geom_errorbar(aes(ymin = lower, ymax = upper),
    width = 0.2, position = position_dodge(0.6), linewidth = 1.5) +
  labs(
    x = "Cover type",
    y = "Predicted mean worker\ncount (per plot)" +
  theme_classic(base_size = 23)

```



Number of workers vs season


```

#create new data grid
worker_data_season<- expand.grid(
  nest_searching_queens_scaled = mean(field_data$nest_searching_queens_scaled, na.rm = TRUE),
  bombus_floral_abun_scaled = mean(field_data$bombus_floral_abun_scaled, na.rm = TRUE),
  cover_type = "herbaceous",
  num_burrows_scaled = mean(field_data$num_burrows_scaled, na.rm = TRUE),
  prop_inactive_scaled = mean(field_data$prop_inactive_scaled, na.rm = TRUE),
  bee_season_type = unique(field_data$bee_season_type)
)

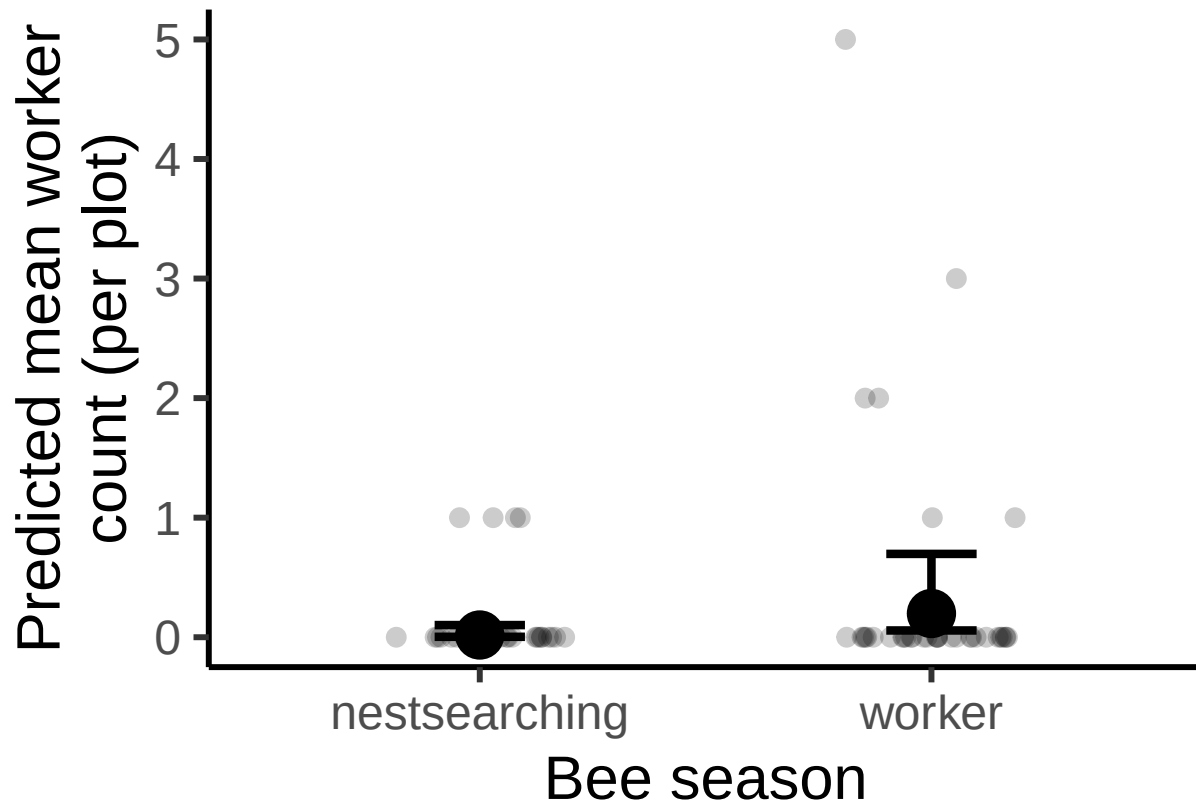
# Get predicted probabilities and standard errors
pred_worker_season <- predict(workers_full,
                             newdata = worker_data_season,
                             type = "link",
                             se.fit = TRUE,
                             re.form = NA)

# Combine predictions with data frame
pred_worker_season_df <- cbind(worker_data_season,
                               fit_log = pred_worker_season$fit,
                               se_log = pred_worker_season$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_worker_season_df <- pred_worker_season_df %>%
  mutate(
    fit = exp(fit_log),
    lower = exp(fit_log - 1.96 * se_log),
    upper = exp(fit_log + 1.96 * se_log)
  )

ggplot(pred_worker_season_df, aes(x = bee_season_type, y = fit)) +
  geom_jitter(
    data = subset(field_data, cover_type == "herbaceous"),
    aes(x = bee_season_type, y = workers),
    width = 0.2,
    height = 0,
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_point(size = 8) +
  geom_errorbar(aes(ymin = lower, ymax = upper),
               width = 0.2, position = position_dodge(0.6), linewidth = 1.5) +
  labs(
    x = "Bee season",
    y = "Predicted mean worker\ncount (per plot)") +
  theme_classic(base_size = 23)

```



Findings

Number of nest-searching queens, number of rodent burrows and the proportion of burrow vacancy does not significantly predict number of workers. Significantly more workers during worker season than during nest-searching season. Significant positive effect of bombus-relevant floral abundance on worker abundance ($p=0.0012$).

Final Path Diagram

